2026 IEEE 11th European Symposium on Security and Privacy (EuroS&P) | 979-8-3195-1074-7/26/$31.00 ©2026 IEEE | DOI: 10.1109/EuroSP68448.2026.00014

SoK: The Engineer's Guide to Post-Quantum Cryptography for Embedded Devices

1st Nikolai Puch
Technical University of Munich
Fraunhofer AISEC
Garching, Germany
nikolai.puch@aisec.fraunhofer.de

2nd Maximilian Pursche
Technical University of Munich
Garching, Germany
maximilian.pursche@tum.de

3rd Sebastian N. Peters
Technical University of Munich
Fraunhofer AISEC
Garching, Germany
sebastian.peters@aisec.fraunhofer.de

4th Michael P. Heinl
Technical University of Munich
Fraunhofer AISEC
Munich University of Applied Sciences HM
Munich, Germany
michael.heinl@hm.edu

***Abstract*—Embedded systems are flexible and cost-effective, and thus ubiquitous, making them attractive targets for cyberattacks. Yet deploying advanced cybersecurity on embedded devices is challenging due to their limited computational power and memory. An emerging risk is quantum computing, which, despite being in its early stages, threatens classical asymmetric cryptography. Given the long lifespan of embedded devices, engineers must urgently address post-quantum security. Post-quantum cryptography (PQC) gained prominence with NIST's standardization initiative for suitable algorithms. During this process, the applicability of PQC to embedded devices became a key research topic. In this work, we survey recent research on PQC for embedded systems, emphasizing applicability to specific use cases rather than focusing solely on PQC algorithms. We provide an overview of the state of the art in PQC and analyze, systematize, and contrast prior work for the three critical embedded use cases: secure communication, secure boot, and secure updates. Overall, PQC already performs competitively with classical cryptography algorithms. However, when performance and memory constraints are an issue, use case-specific considerations should be taken into account when selecting an algorithm for embedded devices. With our findings, we help developers understand the current state of research and its impact from an integration perspective.**

***Index Terms*—Post-Quantum Cryptography, Applied Cryptography, TLS, Secure Boot, Secure Updates**

1. Introduction

Embedded platforms are used throughout today's technology-driven society. They can be found in a wide range of environments, from personal devices to industrial control systems. According to estimates, approximately 16 billion Internet of Things (IoT) devices are deployed worldwide, and this number is expected to continue growing [1]. Despite their functional suitability for many use cases and environments, their security is often inadequate. Due to their long lifespan, their limitation to a few core tasks, and cost pressures, embedded platforms frequently do not receive the same level of attention as traditional IT systems. As a result, they are often vulnerable [2], as demonstrated by multiple large-scale attacks on IoT devices in recent years, such as the Mirai botnet [3]. Thus, although these devices have limited computational capabilities and memory, security must be a priority.

One of the industry's key security challenges is the threat of a cryptographically relevant quantum computer capable of breaking classical asymmetric cryptography. Motivated by recent advances from IBM [4] and Google [5], security research focuses on finding countermeasures. Post-Quantum Cryptography (PQC) is the field of research that seeks mathematical problems that can withstand quantum cryptanalysis. A cornerstone of this research is the PQC competitions [6] held by the US National Institute of Standards and Technology (NIST), which finalized the first three PQC standards in 2024 [7].

The competition raised fundamental questions about the applicability of new PQC algorithms to embedded devices, leading to significant research on embedded PQC in recent years. However, obtaining an overview of the current state of research is challenging due to the rapid developments and the inconsistent integration into applications. This brings up two key questions: What are the strengths and weaknesses of existing PQC algorithms, and which algorithms are suitable for which use cases in the context of embedded devices? To answer these questions, we survey the research on PQC for embedded devices, focusing on the real-world use cases of secure communication, secure boot, and secure updates. Our objective is to provide a consolidated overview to assist developers and integrators in better understanding the challenges they may encounter when implementing PQC for their specific applications. To ensure a reliable representation despite the fast-moving PQC research, this paper focuses on already standardized algorithms and software implementations, as they are more flexibly applicable than specialized hardware.

© 2026, Nikolai Puch. Under license to IEEE.
DOI 10.1109/EuroSP68448.2026.00014

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

2. State of the Art and Related Work

This section provides a brief introduction to the state of the art in PQC. It discusses related work on PQC surveys, PQC in embedded devices, and evaluates our use cases in non-embedded environments.

2.1. State of the Art in PQC

Since the formulation of Shor's algorithm [8], which threatens to break classical asymmetric cryptography, academic research has tried to find quantum-secure alternatives. The field of PQC gained prominence in 2016 when NIST started a competition to standardize PQC algorithms [9]. The competition has just completed Round 4, and the first five standardization candidates have been nominated [10], [11]. A spinoff competition for additional Digital Signature Algorithms (DSAs) is currently in Round 2 [12]. Algorithms offer multiple parameters to meet specific security levels, which are aligned with the effort required to break classical cryptography by a Post-Quantum (PQ) adversary (see Table 9). In addition to the NIST competition, the Internet Engineering Task Force (IETF) also works on standardizing PQC algorithms and related protocols.

PQC has two main primitives: Key Encapsulation Mechanisms (KEMs) and DSAs. KEMs establish a shared secret, while DSAs provide authenticity and integrity. PQ KEMs are intended to replace classical key-agreement protocols, such as Diffie-Hellman Key Exchange (DHKE). The general application flow involves generating an asymmetric key pair, sending the public key to the other party, which then encapsulates the secret and sends the ciphertext back. The initial party then decapsulates the secret using the private key. PQ DSAs are more similar to their classical predecessors and use a long-term key pair to sign and verify messages. With ongoing standardization, a full transition to PQC may be premature. With the rapidly evolving PQC field, early adopters can use a hybrid strategy, especially for KEMs [13], [14]. Hybridization combines classical cryptography and PQC, increasing costs but protecting against PQ adversaries while ensuring conventional security if the PQC algorithm is broken. To narrow down the more than 100 candidate algorithms, we focus on standardized and Round 4 KEMs and DSAs from IETF and NIST. Table 1 gives an overview, and Tables 2 and 3 provide signature, ciphertext, and key size. We use the algorithm names from the NIST trials, rather than the standardized version, for consistency with prior literature. The following subsections group the different algorithms by their underlying cryptographic primitives and explain key performance indicators.

2.1.1. Hash-Based Algorithms. Hash-based algorithms rely on cryptographic hash functions for security. Among all PQC algorithms, they are the most mature. Their well-understood security assumptions make them a good conservative choice for PQC. Because these hashing algorithms are widely used in non-PQ applications, hardware accelerators are available on many platforms, allowing for performance gains [22], [23]. All the mentioned hash-based DSAs support both SHA-2 and Keccak hash schemes and are categorized as either stateful or stateless.

Table 1. PQC ALGORITHMS AND THEIR PUBLICATION STATUS.

Hash-based ciphers		
LMS	stateful DSA	RFC 8554 [15]
XMSS	stateful DSA	RFC 8391 [16]
SPHINCS+	stateless DSA	FIPS 205 [17]
Lattice-based ciphers		
CRYSTALS-Kyber	KEM	FIPS 203 [18]
CRYSTALS-Dilithium	DSA	FIPS 204 [19]
Falcon	DSA	FIPS 206 (WiP [20])
Code-based ciphers		
HQC	KEM	Round 4 selection [21]

LMS and XMSS are stateful hash-based signature algorithms standardized by IETF in RFC 8554 [15] and RFC 8391 [16] and recommended by NIST [24]. Stateful algorithms use One-Time Signatures (OTSs) that utilize a given hash function in a chain. The secret key, signature, and public key are part of this chain. To utilize this technique more than once, these OTSs are combined in a Merkle tree. The final signature consists of the OTS, the OTS public key, and an authentication path along the Merkle tree. Since OTS keys must not be reused, signers must track which keys have been used. The difference between LMS and XMSS lies in their OTS schemes and tunable parameters. The most important parameter is the height of the Merkle tree, which determines the maximum number of signatures. Furthermore, LMS allows adjusting the Winternitz parameter during OTS construction, which impacts both signature size and verification performance [15]. Since large trees are difficult to generate and manage, both XMSS and LMS have a hypertree variant, called multi-tree XMSS [16] or HSS for LMS [15].

SPHINCS+ [25] is the only stateless algorithm, standardized by NIST as FIPS 205 [17]. Stateless algorithms use a Few-Time Signature (FTS) scheme instead of an OTS. SPHINCS+ specifically uses the Forest of Random Subsets (FORS) scheme to sign messages. The FORS is the leaf of an OTSs Merkle tree inside a hypertree structure whose root node serves as the public key. For verification, the signature includes the FORS and a hypertree signature consisting of the OTSs along the authentication path.

2.1.2. Lattice-Based Algorithms. Lattice-based cryptography is among the most promising approaches and thus spawned the most algorithms in the NIST competition [26]. Using lattices for cryptography is a modern approach, which began around the turn of the century [27]. The Shortest Vector Problem (SVP) in high-dimensional lattices and the Learning With Errors (LWE) problem [28] serve as the basis for most algorithms.

CRYSTALS-Kyber is the first KEM that was standardized by NIST after Round 3 as FIPS 203 [18]. It is based on the module LWE problem in cyclotomic rings and uses SHAKE-256 for key generation. Its performance is significantly better than RSA key exchange and can outperform Elliptic Curve Diffie-Hellman (ECDH) in terms of speed [26]. However, compared to ECDH, the key size increases from a few bytes to kilobytes.

CRYSTALS-Dilithium is a PQ DSA originating from the same research collective as Kyber and was standardized as FIPS 204 [19]. It is based on the module LWE and uses the Fiat-Shamir with Aborts [29] technique to

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

generate digital signatures. Overall, it is regarded as the most well-rounded PQ DSA since its sign and verification performance are similar to the Elliptic Curve Digital Signature Algorithm (ECDSA) at lower security levels [26]. The primary disadvantage of Dilithium is its relatively large signature and public key sizes.

Falcon is the last lattice-based algorithm selected for standardization by NIST. In contrast to previous algorithms, it is based on the Short Integer Solution (SIS) problem over NTRU lattices [30], which can be reduced to the SVP for a hardness assumption similar to LWE. Falcon uses a trapdoor sampler called Fast Fourier Sampling [31] to generate signatures. Falcon key generation and signing require 53-bit floating-point arithmetic precision, which is usually not found in hardware on embedded systems. Consequently, Falcon must emulate higher precision operations in software, which slows it down.

2.1.3. Code-based Algorithm. Since most algorithms standardized by NIST are currently lattice-based, NIST started Round 4 with three code-based algorithms: HQC, Classic McEliece, and BIKE. We exclude Classic McEliece and BIKE due to their limited prevalence in embedded use cases, as well as the remaining Round 4 isogeny-based algorithm, SIKE, which is now deemed insecure [32].

HQC (Hamming Quasi-Cyclic) was selected by NIST for standardization [21] as a code-based alternative to Kyber. It is based on error-correcting codes, specifically the Quasi-Cyclic Syndrome Decoding problem. For a code-based algorithm, it offers small public key sizes and precise decryption failure probability. On the other hand, its encryption rate is comparatively low [33].

Table 2. KEY AND CIPHER TEXT SIZE FOR KEMS WITH SECURITY LEVEL 5 IN BYTES. DATA FROM [34].

Cipher	Pub. Key	Priv. Key	Ciphertext
Kyber1024	1568	3168	1568
HQC-256	7245	7317	14 421

Table 3. SIGNATURE AND KEY SIZE FOR SECURITY LEVEL 5 DSAS IN BYTES. ECDSA AND RSA FOR COMPARISON. DATA FROM [34].

Cipher	Pub. Key	Priv. Key	Signature
ECDSA (secp521r1) [35]	66	66	136
RSA 15360 [36]	1920	1920	1920
LMS_sha256_H15_W4	60	64	2672
XMSS-sha2_16_256	64	2093	2692
SPHINCS+-shake-256f	64	128	49 856
Dilithium5	2592	4896	4627
Falcon-1024	1793	2305	1462

2.1.4. Implementations. There are several PQC implementations available, which are utilized in different benchmarks. `PQClean` [37] collects clean standalone *C* implementations of the NIST PQC submissions, which was superseded in 2026 by the `PQ Code Package` [38], a collection of open source implementations within the Linux Foundation. The `pqm4` [39] library optimizes implementations for the Cortex-M4 processor and is regularly updated with changes coming from standardization and NIST round advances. Another project derived from NIST submissions and `PQClean` is `liboqs` [40]. It is the basis for the `OQS-provider` [41], which brings PQC to OpenSSL [42]. `liboqs` is also the basis for an experimental PQC feature in `WolfSSL` [43].

2.1.5. Performance. Figure 1 shows the relative performance of key generation, sign, and verify operations of the five selected PQ DSAs. The data are collected from papers relevant for embedded use cases, which provide data for the primitives as well as for ECDSA [44], [45], [46], [47], [48], [49], [50], [51]. To compare different architectures and libraries, we normalize the results to ECDSA. Not all prior work contains values for the primitives alone, reference algorithms, or all cryptographic operations. Additionally, artifacts are rarely published. Thus, we supplement this dataset with our own results from the `pqm4` benchmark on a NUCLEO-L4R5ZI board with an ARM Cortex-M4, which we extended with ECDSA from the `micro-ecc` library for reference, as well as results from tests with `oqs` on a Raspberry Pi 4 (Pi4) with a Cortex-A72.

Regarding speed, current PQC DSAs perform on par with classical algorithms. For verification, most even outperform ECDSA. However, they exhibit broad dispersion due to differences in CPUs and the ongoing improvement of libraries. We assume more recent implementations perform better. On average, Dilithium is on par with ECDSA and outperforms Falcon, except for verification. Both stateful and stateless hash-based algorithms have similar performance metrics and characteristics. Stateful hash-based algorithms are the next worst, especially for their key generation, which is larger by orders of magnitude. In overall performance, the stateful algorithms beat SPHINCS+ in almost every category, making SPHINCS+ the worst-performing algorithm. While its signing and key generation performance is among the worst of the presented PQ DSAs, their verification performance is decent. Nevertheless, managing the state is a very harsh requirement that limits the possible applications of LMS and XMSS. Memory-wise, PQC DSAs have larger key and signature sizes, especially lattice-based algorithms. Falcon offers smaller signatures and keys than Dilithium. However, compared to ECDSA, its public key is still 27 times larger and the signature is 11 times larger. A distinguishing feature of the hash-based algorithms is their small public key, which is comparable in size to that of ECDSA. The signature size, on the other hand, depends heavily on the height of the Merkle tree and the hash function used. Especially SPHINCS+ signatures can grow very large (>10 kB) due to the FORS authentication path.

2.2. Related Work

This section reviews the existing literature on PQC, focusing on surveys, performance evaluations on embedded devices, and various applications, including Transport Layer Security (TLS) and secure boot.

2.2.1. PQC Surveys. Collecting and categorizing PQC research has been a regular aspect of the NIST competition, as NIST itself publishes a report [26] at the end of each round. Notable other recent surveys in the field of PQC focus on NIST Round 3 algorithms, their improvements, and their implementation in software and hardware [52],

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

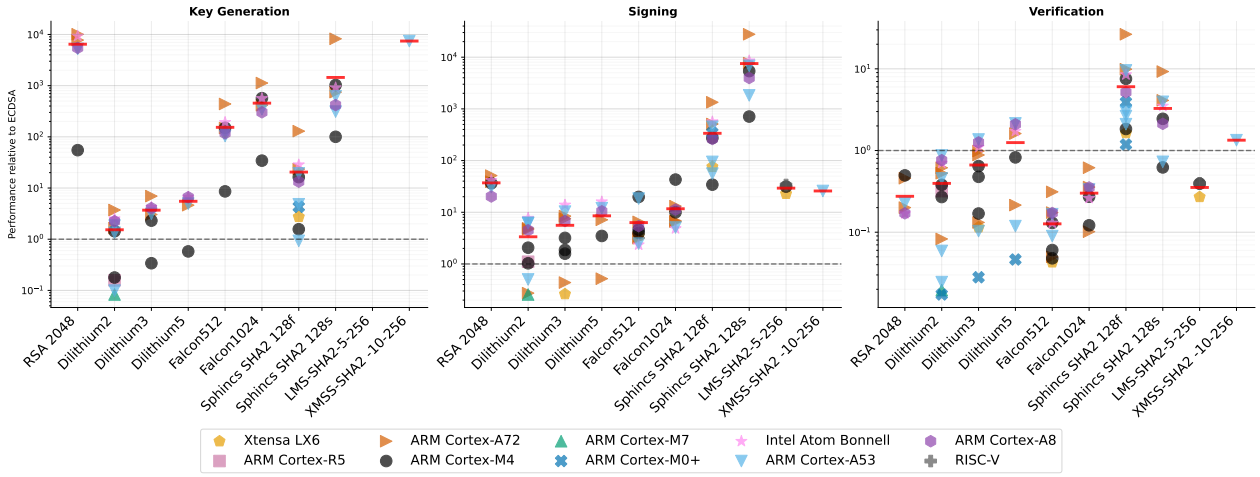


Figure 1. Comparison of different DSAs on different embedded platforms. All results are standardized to ECDSA of the respective run, which is indicated with the horizontal dotted line. The horizontal red line — marks the average for one algorithm.

[53], [54]. They highlight some case studies, provide high-level recommendations, and identify challenges, but do not offer insights for specific use cases. Shim [55] extends this work by compiling metrics for multiple DSAs across various systems to assess their applicability to protocols such as TLS, SSH, IPsec, and DNSSEC in IoT. The migration and standardization process itself, grouped by keyword and application field, was investigated by Alnahawi et al. [56]. The IETF is working on a draft for PQC for engineers [57], which gives a comprehensive introduction to the cryptographic implications of quantum computing, the NIST and ISO PQC and hybrid algorithms, security considerations, and transition strategies. However, it delves deep into cryptography but lacks actionable, use-case-specific recommendations, e.g., for secure boot. Details for constrained environments are lacking, even though a second IETF draft [58] aims to address this topic.

2.2.2. PQC on Embedded Devices. Early work on benchmarking and surveying NIST PQC algorithms on constrained hardware focused on the cost profile of algorithms rather than more complex use cases [59], [60], [61]. For example, Hession et al. [62] extended the XXBX framework in 2019 to run tests on an ARM Cortex-M4F. Even at this early stage, lattice-based algorithms have shown promise, a trend that continues to this day [63]. However, a major limiting factor for Falcon is the lack of 53-bit floating-point precision in Microcontroller (µC), which becomes evident when comparing benchmarks on the ARM Cortex-M7 and Cortex-M4. Falcon improves in all operations except verification on the double-precision floating-point Cortex-M7, closing the gap to, or even outperforming, Dilithium [64]. Even though performance is promising, memory requirements are often significantly higher, especially on constrained industrial embedded devices [48].

There are more possible PQC use cases than the ones described in this work. For example, PQC may get used in a message brokering system that operates on Kubernetes and is based on Kyber-ASCON [65]. Instead of message brokering, publish-subscribe via MQTT [66], [67] or OPC UA [68] is often used in energy or industrial use cases. Another use case is authentic unmanned aerial vehicle communication, where PQ DSAs performance on embedded devices is dominant [69], [70]. Finally, research investigates quantum-safe electronic identity and travel documents [71]. But chip cards are often highly constrained and use specialized protocols, making lessons learned from PQC application difficult to transfer to other embedded devices. For this reason, we do not consider this use case in our survey, but encourage interested readers to consult their work for details specific to this use case.

Vehicular security is a key use case for PQC. It uses embedded µCs such as ARM Cortex-A72 or A7, as well as purpose-built controllers. The Infineon AURIX architecture is a typical mid-range example, with the TC39x running 300 MHz and providing 16 MB of flash and 2816 kB of SRAM. In parallel to the trials, the NIST algorithms have been evaluated on such µC [72], [73], [74], [75], with Kyber showing early promise independent of a specific use case [72]. DSAs present a more nuanced picture. Their larger key and signature sizes increase the burden on automotive key management, particularly given µC memory constraints and low-bandwidth networks such as CAN. While Dilithium provides better overall performance, Falcon offers more favorable trade-offs in verification-heavy scenarios, such as secure boot or secure updates [74], [75]. However, the Falcon implementation is comparatively large, though verification-only variants can reduce its size to that of Dilithium [75]. Further work proposes a PQC Authenticated Key Exchange (AKE) to establish secure communication on an AURIX platform [76]. They combine Newhope with an XMSS-based Public Key Infrastructure (PKI) and ran performance measurements on their proposal. Due to the low ISO-TP over CAN data rate of 500 kbit/s, data transfer becomes the dominant cost, rather than cryptographic computations, for the AKE.

2.2.3. PQC Use Cases on Non-Embedded Devices. Our selected use cases are also crucial in non-embedded, traditional IT environments, motivating substantial research. Although they do not share the same constraints, the results still provide important context for our analysis.

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

TLS is the most common form of secure communication, with most online traffic utilizing it through HTTPS [77]. As a result, the application of PQC in TLS has been extensively examined. The NIST Round 2 and 3 submissions have been extensively evaluated for TLS 1.3 [78], [79], [80], [81]. They find the selected KEM has less impact than the DSA, with Kyber and HQC performing on par with Elliptic Curve Diffie-Hellman Ephemeral (ECDHE). Falcon performs best if sufficient floating-point precision is given. Otherwise, Dilithium is also competitive with classical cryptography. On the server side, signing costs become magnified across many connections, as discussed in detail by Kampanakis et al. [82]. Due to Falcon's poorer signing performance, Dilithium is preferred in this context. However, future optimized implementations and hardware acceleration are expected to enhance server performance. In 2024, Alnahawi [83] published a survey on PQ TLS that also briefly discusses embedded devices. While their findings on the overall solid performance of PQC in TLS align with our results, they primarily focus on ideal network conditions, which are rare in real-world embedded use cases. When looking at the transition to PQ TLS in Android, it is notable that many apps open numerous connections (median 94) [84]. Most of these connections are to the same host and do not utilize session resumption. This must be taken into account, as it can magnify potential cryptographic overhead or even negate potential gains of a careful algorithm selection. Additional benefits can be obtained by using KEMTLS to reduce overhead and KEMTLS-PDK if long-term keys are available.

3. Use Cases for Embedded Devices

For the use cases, we focus on typical embedded µCs and Microprocessors (µPs). The ARM Cortex-M4 is a typical example of a 32-bit µC in the mid-range segment of embedded hardware, as well as the reference platform chosen by NIST [90]. It has typically 16 – 640 kB of RAM, up to 2 MB of flash, and operates at a maximum clock speed of 72 – 180 MHz [91]. As an example, we select the current `pqm4` default core, the L4R5ZI (cf. Table 4). The Cortex A53 is an example of a µP we consider, as it is used, for example, in many automotive applications. We focus less on more powerful µPs, as their characteristics more closely resemble those of low-power Personal Computers (PCs), diminishing the effects of PQC operations. Lower-end µCs struggle with cryptography and require individual consideration. Especially the special use case of smart cards, often supplemented with dedicated cryptography co-processors, merits a separate analysis. The different performance data of the cores are given in Table 4.

We describe three use cases to determine the suitable PQ algorithms for this hardware. The main use cases for asymmetric cryptography in IoT are secure network communication, trusted execution via secure boot, and secure updates. We specifically analyze TLS for secure network communication and Software Updates for Internet of Things (SUIT) for secure updates. Table 5 presents the different applications of asymmetric cryptography within these use cases.

3.1. Secure Communication

TLS is the de facto standard for secure web communications [92], providing an end-to-end secure channel over TCP with authenticity, data confidentiality, and message integrity. Its flexibility in cryptographic algorithms makes it popular in embedded environments [93]. Traditionally, embedded platforms act as clients connecting to management systems, but especially in IoT, they increasingly act as servers, such as smart home bridges or configuration interfaces. The latest version, TLS 1.3 [94], standardized in 2018, offers several improvements, yet TLS 1.2 remains common in embedded systems due to their longevity and upgrade challenges.

3.1.1. TLS Process. A TLS session consists of two phases. The first phase is the *handshake protocol*, which authenticates the parties and agrees on cryptographic algorithms. X.509 certificates are used to authenticate the server and, optionally, the client. During this phase, a shared secret is negotiated to derive the subsequent transport keys. In the following *record protocol* phase, these transport keys are used to symmetrically encrypt the application data.

We begin by outlining the classic TLS handshake to highlight the differences compared to PQ TLS, illustrated in Figure 2. In TLS 1.3, the client first sends a *ClientHello* message that includes the supported cipher suites and the client's key share. The server selects a cipher, calculates the master key, and responds with a *ServerHello*. The message contains the server's key share and is followed by the server's certificate. If mutual authentication is required, the server requests a client certificate via a *CertificateRequest* message. The size of the individual certificates and the length of the certificate chain are the primary factors that determine the amount of data transmitted during the handshake. Certificate verification is usually the most expensive computational operation in the TLS handshake, especially when multiple signatures in the certificate chain need to be verified. Based on the chosen key exchange, both sides calculate the shared secret, conventionally using DHKE. From this point forward, all messages are symmetrically encrypted using session keys derived from the shared secret.

Asymmetric cryptography vulnerable to PQ adversaries is only used in the *handshake protocol*. More precisely, it is present in the DHKE and the X.509 certificates. For PQ, a KEM is used for the key exchange, which is the most critical part, as a KEM that becomes vulnerable in the future may allow harvest-now, decrypt-later attacks. Here, the client generates an ephemeral key and transmits its public key as the key share in the *ClientHello*. The server uses it for an encapsulation to get the shared key and the corresponding ciphertext. It sends the ciphertext along with the *ServerHello*, which can then be decapsulated by the client. For the X.509 certificates, a PQ DSA must be used.

3.1.2. Embedded PQ TLS Literature Analysis. In this section, we present our findings from our analysis of the prior literature on quantum-secure communication on embedded devices. Table 6 gives an overview of all the papers referenced in this section. Most research is based on either

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

Table 4. TECHNICAL DATA OF REPRESENTATIVE PROCESSORS, INCLUDING THEIR ARCHITECTURE, CLOCK RATE, AND MEMORY. WE FOCUS ON THE MID-RANGE CORES LIKE CORTEX-M4, A53, AND LX6.

	ATmega328 [85]	**Cortex-M4** [86]	**LX6** [87]	**Cortex A53** [88]	Cortex A72 [89]
Type	µC	µC	µC	µP	µP
Architecture	8 bit	32 bit	32 bit	64 bit	64 bit
Clock Rate	16 MHz	120 MHz	240 MHz	1.2 GHz	1.5 GHz
RAM	2 kB	640 kB	520 kB	1 GB	8 GB
DevBoard	Arduino Nano	NUCLEO-L4R5ZI	ESP32-D0WD-V3	Rasp. Pi 3+	Rasp. Pi 4

Table 5. CRITICAL KEM AND PQ DSA FOR THE USE CASES FROM THE DEVICE PERSPECTIVE. FOR TLS, FIRST THE CLIENT AND THEN THE SERVER PERSPECTIVE IS LISTED.

		TLS	Secure Boot	SUIT
KEM	Key Generation	✓	x	x
	Decapsulate	✓ / x	x	x
	Encapsulate	x / ✓	x	x
Signature Algorithm	Key Generation	x	x	x
	Verify	✓ / x[1]	✓	✓
	Sign	x[1] / ✓	x	x
	Long-term Key	x	✓	✓

[1] if without TLS client authentication

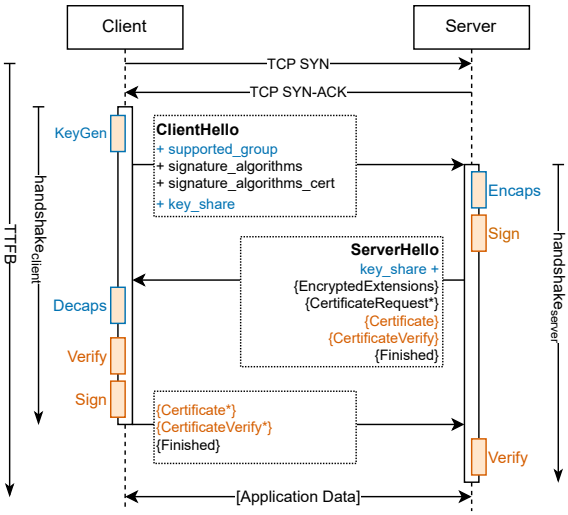


Figure 2. Mutually authenticated PQ TLS handshake. Items in blue correspond to the KEM, those in orange to the DSA. * denotes optional client authentication. {} and [] denote encrypted messages.

the ARM Cortex-M4 or Cortex-A53. As a library, *pqm4* is most prevalent, which is incorporated into either WolfSSL or mbedTLS as the TLS library to support TLS 1.3. Newer experiments on more performant hardware often opt for the more convenient Open Quantum Safe (OQS) OpenSSL library. Algorithm-wise, most researchers use the NIST candidates current at that point in time. This means that Kyber, Dilithium, and Sphincs+ are analyzed in almost all prior work. Stateful hash-based algorithms have received the least attention, with only Paul et al. [95] investigating XMSS.

Impact of KEMs: The choice of KEMs has less impact compared to the selected DSA and the network bandwidth [51]. Kyber outperforms ECDHE in terms of computation speed on the Cortex-M4 [44], [93], while only marginally adding to the handshake bandwidth. HQC encapsulate time is only 30 ms slower than Kyber, but the communication overhead of the larger public key and ciphertext results in a 1.7 s slower handshake via Bluetooth Low Energy (BLE) [93]. The results for the Raspberry Pi 3B+ (Pi3B+) show slightly better latency and energy consumption for Kyber and SABER than ECDHE [50]. As Kyber is currently the only standardized KEM, early adopters may have less incentive to implement other options, regardless of network conditions or topology.

Impact of DSAs: On the other hand, the selection of the DSA largely determines TLS performance and is more dependent on external factors, such as network conditions. This is due to the increase in certificate size for all PQ algorithms, which results from their larger signature sizes compared to ECDSA (see Table 3). The increased size, then, in turn, leads to higher network bandwidth requirements for the handshake. For typical PKI architectures, the size increase affects the overall handshake size multiple times, since DSAs are present in each transmitted certificate.

Many recommendations focus on Falcon and Dilithium, mainly due to the bad performance of SPHINCS+. Especially when used as a server or for mutual authentication, embedded devices are slow in comparison, taking over 10 – 50 times longer than the classical handshake, due to the slow SPHINCS+ signing process and large signature sizes [45], [47]. This trend continues on more powerful embedded platforms, where ECDSA still performs 3 – 8 times better than SPHINCS+ [50], [96]. In scenarios with server-side authentication only and high bandwidth, SPHINCS+ can still be acceptable, due to its reasonable verification performance. Bürstinghaus-Steinbach et al. [47] showed that PQ handshake times for embedded clients are very similar in these conditions to those with classical crypto, even outperforming the ECDHE-ECDSA baseline in some cases. This is largely due to the good performance of Kyber in comparison to ECDHE. However, they found SPHINCS+ may not be suitable as a PQ DSA for embedded devices that require message signing. Thus, the main reason for SPHINCS+ remains its conservative security assumptions.

The primary tradeoff between Dilithium and Falcon is bandwidth versus signing performance. Dilithium provides fast signature computation and verification, but it significantly increases the handshake size compared to Falcon. This means that combinations with Dilithium-Kyber are very competitive with ECDHE-ECDSA in high-bandwidth environments [50], [95], even outperforming it on security level 1 [45]. On the other hand, size can have a significant impact on low-bandwidth network connections, as shown by Schöffel [93] and Gonzalez et al. [97]. For the BLE connection, handshakes with Dilithium certificates were slower, as they spent most of their time transmitting and receiving data due to the constrained network bandwidth [93]. Falcon, on the other

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

hand, performs well in restricted network environments and if no client authentication is required, due to its decent verification performance and small signature size [93], [97]. Combinations with Falcon and Kyber take twice the handshake time on the Cortex-M4 but only need to transmit half the communication size [45]. It suffers from a lack of hardware support for double-precision floating-point, which makes its signing operation perform poorly. This effect is diminished on more performant cores, such as the ARM Cortex-A53 or A72, where Falcon-Kyber outperforms Dilithium in both the lower bandwidth BLE and WiFi scenarios [96].

A special case is hybrid TLS, which combines both classical and PQ algorithms to remain secure if either one is broken. Marchsreiter et al. [51] investigated this with the NIST 3rd Round finalists on PC and Pi4. ECDSA+Dilithium-NTRU performs best, with Kyber close behind due to the KEM only having a minor impact. Falcon performs overall worse, which in turn lessens the effect of Hybridization. In general, the overhead of hybrid algorithms increases with higher security levels. E.g., ECDSA+Dilithium2 is 1.4 times slower, which increases to 8.3 for Dilithium5 compared to their only PQ counterparts [51]. When examining hybrid KEMs in constrained networks alone [98], Kyber experiences a $12.4\,\%$ increase in latency and a $32\,\%$ increase in energy consumption compared to non-hybrid mode. This effect is reduced for the generally more expensive KEMs. For example, HQC, whose non-hybrid handshake is approximately 2.6 times slower than Kyber, has a hybrid handshake latency overhead of $5.9\,\%$.

Impact of PKI Structures: On the one hand, the DSA's impact is amplified in multi-level certificate hierarchies. On the other hand, this allows for clever arrangement of a PKI structure to balance out algorithms or meet specific requirements. Schöffel et al. [93] tested mutually authenticated TLS with a two-layered, mixed certificate chain PKI with different algorithms for client, server, and Certificate Authority (CA). To emulate a distributed sensor network, they connected the battery-powered Cortex-M4 to a gateway via BLE, which then connects to the server via Ethernet. Due to this setup, high bandwidth requirements are punished and algorithms with large signatures, public keys, or ciphertexts perform worse. Using Falcon for all certificates almost halves handshake time compared to pure Dilithium, due to its lower transmission delay, making it the best-performing pure PQC combination. However, in this setup, PQC cannot outperform ECDHE due to its communication overhead. Paul et al. [95] evaluated the impact of mixed DSAs certificate chains on TLS 1.3 on both embedded and PC platforms. They connect a Raspberry Pi 3B (Pi3B) to remote servers running on different Azure virtual machines and measure Time To First Byte (TTFB), client and server handshake times, communication size, the library's code size, and peak memory usage. They argue that hash-based algorithms, being based on well-established cryptographic primitives, are especially suited for root CAs, where a broken key is especially critical, fewer keys need to be generated, and fewer signing operations are required. For the end entity, faster, smaller lattice-based algorithms should be used, resulting in XMSS-Dilithium-Kyber as the best-performing pure PQC algorithm under these constraints. To sum up, mixed PKIs might perform best in the embedded use case. Using Falcon for server authentication and CA certificates, while deploying Dilithium certificates to the embedded clients, appears to yield the best of both algorithms.

The effect of PQC on certificate revocation via Certificate Revocation List (CRL) and Online Certificate Status Protocol (OCSP) stapling was investigated by Hanna et al. [96]. They evaluate impact on TLS handshake time, computational overhead, memory footprint, and energy consumption. Their evaluation platform is an ARM Cortex-A72, connected via either BLE or WiFi to a Gateway, which in turn is connected to the server. The CA is hosted in the cloud and supports hybrid signatures. While CRL outperforms OCSP for all algorithms and network scenarios, Hanna et al. still recommend OCSP due to size and storage restraints. Additionally, the computational overhead can be reduced by caching.

Impact on Power Consumption: Power consumption in embedded devices is largely determined by computational costs. Thus, in their investigation of energy and power consumption for PQ TLS, Tasopoulos et al. [44] found that PQC, except SPHINCS+, is also competitive with ECDSA in terms of power consumption. Using mixed DSAs, as described above, can be used to optimize power consumption [93]. A pure Falcon PKI architecture is dominated by signature computation rather than transmission delay. Computation on the embedded device impacts battery consumption more than transmission, making Falcon use more energy even though it completes the handshake faster. However, Falcon for CA and server certificate and Dilithium for the client reduces power consumption by $36\,\%$, while only slowing down performance by $14\,\%$ compared to a full Falcon PKI [93].

Impact on RAM: Some papers also measure RAM usage during the handshake [45], [47], [99]. The memory of embedded devices is limited, which may restrict the algorithms they can run. For example, Classic McEliece cannot be implemented in many µCs due to memory limitations. For the Cortex-M4, PQ TLS implementations require, depending on the algorithm and security level, $43 - 85\,\mathrm{kB}$ of RAM, which is $18 - 35$ times more than the average Elliptic Curve (EC) TLS implementation [45]. The notable exception is Sph1-Kyb1, which only requires $800\,\mathrm{B}$ or only a third of the EC implementation. However, the ARM Cortex-M4 typically has $640\,\mathrm{kB}$ of RAM and the Cortex-A53 supports up to $4\,\mathrm{GB}$ of RAM, allowing sufficient space for customer applications.

Secure Communication Special Cases. While we decided against evaluating alternatives to TLS, special use cases of TLS, such as charging, are still under consideration. Modern Electric Vehicle (EV) charging allows charge planning and billing via ISO 15118, which utilizes single-sided TLS to secure communication. With their proposal *QuantumCharge*, Kern et al. [99] investigated how to move this standard to PQC. It uses Kyber instead of ECDHE and considers the NIST algorithms Dilithium, Falcon, and SPHINCS+. Still, the framework is designed to allow for later algorithmic changes. They evaluate it on a Pi3B+ connected via Power Line Communication, as it is comparable with both EV and Charging Point (CP) hardware. Their results show good performance for Dilithium and Falcon, but SPHINCS+ can also meet the

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

timing requirements of ISO 15118. However, due to the larger signature size of all PQC algorithms, the maximum certificate size in ISO 15118 would need to be increased.

KEMTLS is an interesting field of research for embedded platforms since it can reduce memory requirements and handshake bandwidth. Gonzalez et al. [97] focus on KEMTLS. KEMTLS was first proposed by Wiggers et al. [100] and is an alternative implementation of PQC into TLS 1.3. Instead of PQ DSAs, KEMTLS uses KEMs for authentication during the handshake protocol. This is achieved by establishing long-term KEM public keys and distributing them via a PKI. CAs still utilize PQ DSAs within the PKI itself to sign messages, but the end-entity certificates contain a KEM public key. A party can prove its authenticity by decapsulating a secret that the other party encapsulated with the respective public key. This secret is incorporated into the shared secret. The main motivation behind KEMTLS is to improve performance and simplify the protocol by minimizing the number of required PQC algorithms during the TLS handshake. Since KEMs are needed to establish the shared secret for symmetric encryption anyway, using the same KEM in authentication simplifies the code base and reduces complexity. Furthermore, most KEMs perform encapsulate and decapsulate operations faster than the PQ DSAs sign and verify. The authors evaluated different KEMTLS combinations on the Cortex-M4 and compared them to PQ TLS using PQ DSAs for authentication. Their implementation builds on the WolfSSL library. Their test setup includes a two-layered PKI hierarchy with a PQ DSA CA certificate and a KEM server certificate. In the conducted KEMTLS and PQ TLS experiments, only the high-powered server is authenticated. The authors simulate three different network conditions: regular broadband with 1 Mbit bandwidth and 26 ms Round-Trip Time (RTT), LTE-Machine Type Communication (LTE-M) with 1 Mbit bandwidth and 120 ms RTT, and Narrowband-IoT (NB-IoT) with 46 Kbit bandwidth and 3 s RTT. As KEM, SABER performs best, being less than one percent faster than Kyber. Ignoring Rainbow combinations, Falcon is again the best-performing PQ DSA and outperforms KEMTLS under all network conditions. While KEMTLS with Kyber-Dilithium is on average $16\,\%$ faster than PQ TLS with the same algorithms, PQ TLS is in turn still $10\,\%$ faster than KEMTLS with Kyber-Falcon. This effect is the smallest for NB-IoT, where the difference is less than $1\,\%$. Regular PQ TLS benefits most from deploying Falcon both as PQ DSA in the CA and server certificate. Since the embedded platform does not need to sign the handshake, Falcon's fast verify operation and small public keys and signatures significantly speed up the handshake compared to Dilithium. Regarding handshake traffic amount, KEMTLS and PQ TLS are roughly equal in size and perform similarly in the given setups. While in Gonzalez et al.'s experiments [97], KEMTLS does not outperform the regular PQ TLS with Falcon and SABER/Kyber, they test only with server-side authentication. However, mutual authentication would significantly impact PQ DSA performance due to the expensive sign operation. Since KEMTLS uses the KEM encapsulation operation to authenticate, which performs well on embedded devices, another comparison with mutual authentication would be interesting.

3.1.3. Findings. Overall, we find that the methodologies are difficult to compare. First, due to the evolving nature of PQC, the tested algorithms and the libraries vary across publications. For example, while HQC is currently being standardized by NIST, it is seldom evaluated, a gap that future work should address. Additionally, not all studies evaluate a classical algorithm such as the Rivest–Shamir–Adleman cryptosystem (RSA) or ECDSA, which reduces comparability and removes context. Moreover, the notation is not always clear, particularly with respect to whether SPHINCS+ is used with SHA or SHAKE. However, this distinction can significantly affect the results, especially on platforms where hardware accelerators are present. Artifacts such as complete measurements are rarely published, which complicates meta-analysis. A trend can be observed where, for more constrained devices, only the primitives are evaluated. In contrast, more complex use cases are evaluated on more powerful hardware, usually a $64\,\mathrm{bit}$ µP. Finally, one should note that, for use cases with high application payload or few connection establishments, efficiency is driven by data economy in the use case rather than the selected cryptography. Here, an emphasis on security rather than efficiency when selecting an algorithm is valid.

In summary, we find that DSA selection and network conditions dominate the performance of embedded PQ TLS. This is caused by the significant increase in key and signature sizes. Currently, for DSAs, a tradeoff between performance and network load must be weighed (see Figure 5). Here, a mixed certificate chain PKI or KEMTLS might prove useful to balance the algorithms. Future work may explore KEMTLS for mutually authenticated connections or the impact of hybrid PQC on embedded devices. We propose that future work clearly define measures, e.g., latency as either the number of full TLS connections in a given time frame or the TTFB. Additionally, at least one classical algorithm should be tested, preferably ECDSA-ECDHE, as RSA generally performs poorly.

3.2. Secure Boot

Secure boot on embedded devices aims to restrict code execution to legitimate software by verifying each software image through digital signatures. Naturally, this involves multiple stakeholders from chip producers to embedded solution vendors.

3.2.1. Secure Boot Process. The full authentication process of the secure boot chain is visualized in Figure 3. It starts with the execution of a trusted Zero-Stage Boot Loader (ZSBL) located in Read Only Memory (ROM). On embedded platforms, this bootloader establishes a chain of trust by loading the next firmware image from flash memory into the internal RAM, authenticating it, and passing the control flow to it. Authentication is based on the certificate attached to each software image, a signature over the image itself, and corresponding metadata. To minimize the impact of firmware size on verification, the firmware is usually pre-hashed before validation. The initial stage of this chain, which resembles the Root of Trust (RoT), is particularly critical. This trust is derived from a bootloader public key whose hash is burned into the One Time Programmable memory (OTP memory) of

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

Table 6. SUMMARY OF THE PQ TLS PAPERS. RUNTIME IS MEASURED ON THE BOARD AND MESSAGE SIZE REFERS TO THE TLS HANDSHAKE. HANDSHAKE (HS) TIME ARE MEASURANDS DEFINED IN FIGURE 2 AND LATENCY IS AN UNSPECIFIED DELAY CAUSED BY TLS.

Reference	Measurand								Algorithm													Impl.	Platform	Network Topology	PKI Structure
	Msg. size	Storage	Memory	HS Time	Latency	Runtime	Power	Energy	Kyber	HQC	SABER	Bike	Dilithium	SPHINCS+	Falcon	NTRU	Frodo	XMSS	ECDHE	ECDSA	RSA				
Bürstinghaus et al. [47]	■		■			■			■					■					■	■		Ref. impl., mbedTLS	Cortex-A53, LX6 dual-core, ARM966E-S, Cortex-M0+	Ethernet; Direct conn.	Single pre-shared certificate Server authentication
Tasopoulos et al. [45]	■		■	■					■	■	■	■	■	■	■	■			■	■	■	pqm4 WolfSSL	Cortex-M4	Ethernet; Direct conn.	Homogeneous 2-tier hierarchy Mutual authentication
Tasopoulos et al. [44]				■			■	■	■	■		■	■	■	■		■		■	■	■	pqm4 WolfSSL	Cortex-M4	Ethernet; Direct conn.	hierarchy unstated, Server and mutual auth.
Schöffel et al. [93]			■		■			■	■	■	■	■				■	■		■	■	■	pqm4, Ref. impl., mbedTLS,	Cortex-M4, Cortex-A53	BLE; Gateway conn.	Mixed 2-tier hierarchy Mutual authentication
Gonzalez et al. [97]	■		■	■		■			■		■		■		■	■						pqm4, WolfSSL	Cortex-M4	BB / LTE-M / NB-IoT	Mixed 2-tier hierarchy Server authentication
Halak et al. [50]			■		■			■	■		■		■	■					■	■		libOQS, mbedTLS	Cortex-A53, Cortex-M0+	Ethernet/WiFi; Direct conn.	hierarchy unstated, Mutual authentication
Kern et al. [99]	■	■		■					■				■	■	■					■		OQS OpenSSL	Cortex-A53	PLC; Direct conn.	2 Sub-CAs, emulating max. ISO 15118 PKI architecture
Paul et al. [46]	■		■	■					■				■	■	■			■	■	■		Ref. impl., WolfSSL	Cortex-A53	Ethernet; Remote server	Mixed Certificate Chains Mutual authentication
Hanna et al. [96]			■	■			■		■				■	■	■				■	■	■	OQS OpenSSL	Cortex-A72	BLE/WiFi; Gateway conn.	Interm. CA with CRL and OCSP Server and mutual auth.

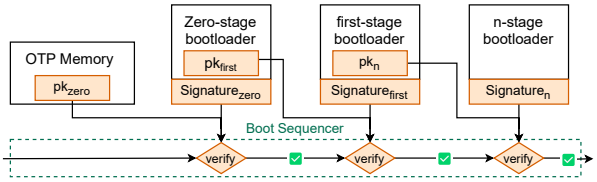


Figure 3. Exemplary secure boot validation chain with all parts impacted by the PQC migration highlighted in orange. pk denotes a public key.

the chip during manufacturing. The subsequent bootloader images are signed with the corresponding private key and are verified by the previous bootloader. Once the control flow passes all the bootloader images, the firmware is running and can initialize additional hardware or start customer software. Since other entities usually develop the software rather than the firmware, different key pairs are used to sign it. The public keys are part of the firmware and thus implicitly authenticated in the chain.

Secure boot imposes certain restrictions and requirements on the signature algorithm in use. The longevity of public keys is the primary consideration. The chip manufacturer generates and manages the key pair for the ZSBL, which is typically used for an entire chip revision. As the public key is burned into each chip, it cannot be replaced during its lifetime. Therefore, the security level of the signature algorithm should be chosen conservatively to accommodate advances in computing power and cryptanalysis. At the same time, the rapid advances in PQC algorithms complicate decisions for such a long product lifespan. The second restriction is performance. Signing and key generation performance are less critical in this case, as they are performed during the chip's manufacturing rather than during the boot process itself. On the other hand, signature verification directly affects boot times and is its most important performance factor. Especially for embedded systems with safety functionality, boot time can be crucial. Slow verification repeatedly impacts boot time, since each step has its own signature that needs to be verified. Finally, the main limitation on embedded platforms is memory, particularly the internal on-chip RAM size. This is especially critical for µCs like the Cortex-M4, which has only 192 kB – 640 kB of RAM. The bootloader images, their corresponding signatures, the bootloader public key, and the verify code itself are loaded into this internal RAM. Since the CPU can only use internal RAM during the first stages of the bootloader, there is a hard limit on memory consumption. Thus, a PQC algorithm must have minimal signature and public key sizes, as well as minimal stack usage during verification. After boot, the firmware can initialize external RAM, easing memory constraints for customer software.

3.2.2. PQ Secure Boot Literature Analysis. This section analyzes the aforementioned restrictions for multiple embedded platforms. An overview of the mentioned papers can be found in Table 7.

Selection of the Algorithms Class: The first finding is that current research primarily focuses on hash-based algorithms for the secure boot use case. Algorithms should be chosen conservatively since the RoT in the ZSBL is immutable. Under these circumstances, hybridization could be highly beneficial. However, the strict hardware and timing requirements of secure boot make it infeasible on many constrained embedded devices. Given the long lifespans of embedded devices, this further supports the use of hash-based algorithms, whose security properties are well understood. Furthermore, stateful hash-based algorithms offer good verification performance and relatively small signatures and public keys, which is crucial for constrained embedded devices.

In particular, the two stateful algorithms, LMS and XMSS, attracted attention, likely also because NIST recommends them for firmware authentication [24]. The main disadvantage of managing state is considered less

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

problematic, since key generation and signing are performed by the manufacturer and firmware developer, who are not constrained by the embedded platform. Further information on state management can be found in the NIST recommendation [24] and the work of McGrew et al. [101]. With secure boot, only a limited number of signatures needs to be generated under very strictly defined circumstances, such as the release of new firmware. This makes it easy to centrally manage the state. Software implementations of LMS generally outperform XMSS on embedded platforms [22], [102]. Compared to RSA and ECDSA, the stateful algorithms outperform the classical algorithms, while SPHINCS+ performs worse [22]. Additionally, general-purpose hash accelerators can significantly improve the performance of hash-based DSAs, even without specialized additions [22]. This is something we will discuss in greater detail later.

However, lattice-based algorithms offer better verification performance. For example, Marzougui et al. [103] have already analyzed the former NIST Round 2 candidates on an ARM Cortex-R5 in 2019. They found that qTESLA, a lattice-based algorithm later dropped by NIST but with very similar performance characteristics to Dilithium, achieved verification performance that was almost five times faster than XMSS.

Secure boot with Dilithium was analyzed by Bos et al. [104]. For this, they integrated a custom low-memory Dilithium version into the secure boot process of an NXP S32G vehicle network processor, which consists of an ARM Cortex-M7 and Cortex-A53 cores. The S32G's Hardware Security Engine (HSE) has a feature that uses a reference proof for consecutive boots and only verifies the signature during initial installation. For this initial installation, they compared their Dilithium implementation with RSA and ECDSA, which were $5-10$ and $2-5$ times slower, respectively. But overall, the slowest boot with Dilithium5 was still only $26.6\,\mathrm{ms}$, which may often be acceptable. Additionally, they showed the significant impact of the pre-hashing step, as verification of the larger firmware is approximately an order of magnitude slower.

Considerations for hash-based Algorithms: When comparing LMS and XMSS, LMS offers a wider range of parameters to choose from, particularly the flexible Winternitz parameter that affects the size and verification performance of the OTS. Software implementations of LMS struggle with large Winternitz parameters, resulting in a 20-fold increase in verification run time between the lowest (w = 4) and highest (w = 256) parameters [22]. Despite this, LMS still outperforms XMSS even with higher Winternitz parameters. Tree size is an important parameter that is handled differently by the presented papers. In the single-tree variants of LMS and XMSS, the tree height must be sufficiently large to generate enough signatures throughout the key's lifetime. Final height depends on the device's update rate, as well as the number of firmware images in the secure boot chain. The smallest tree height considered in the presented papers is 10, which performs best but only allows $2^{10} = 1024$ signatures, which might not suffice for the entire lifespan of the device. The next standardized height is 15 for LMS and 16 for XMSS, which can generate a more flexible number of signatures with $2^{15} = 32768$ or $2^{16} = 65536$. It is essential to choose the tree height with foresight, particularly for keys used during the initial stages of secure boot, to ensure that there are sufficient signatures available. This is because the public key present in the ZSBL cannot be changed, limiting the total number of signatures that can be used throughout the device's lifetime.

The rapidly verifiable XMSS signature approach [105] shows promising performance improvements for stateful algorithms. For this, the signer appends random counters to the message until they find a shorter OTS hash chain during verification. The approach's results demonstrate that verification speed increases by $30\,\%$, even with a low number of counters tried, regardless of the Winternitz parameter of LMS on software and general-purpose accelerators [22]. Since the additional effort affects only the firmware provider but unburdens all devices during every boot process, the approach should be pursued for stateful signatures.

Considerations for Secure Boot Flow: The secure boot flow itself can be modified to optimize for a specific algorithm. Román et al. [106] propose a new two-stage boot process. The first authentication check performed uses a Physical Unclonable Function (PUF) that relies on SRAM cells instead of a signature and an unalterable public key. The ZSBL checks the first image by computing a Hash-based Message Authentication Code (HMAC), which relies on a symmetric key generated from a PUF based on the startup values of the device's SRAM cells. SRAM cells have either random or static values after boot, which vary from device to device but remain fixed after manufacturing. This method can be used to generate a device-specific key, which is then utilized for symmetric HMAC authentication. Once the ZSBL verifies the first image, the control flow is passed to the next bootloader, which then loads and verifies the firmware and application software. The authors propose using XMSS for this step. However, they suggest using only the Winternitz OTS to verify the firmware, since larger Merkle tree heights lengthen the authentication path and slow verification. The authentication path from the OTS key to the Merkle tree root can be verified once at purchase time and after each update, proving authenticity while lessening the burden on the device. Implementation of this proposal is benchmarked on an ESP32 µC equipped with a SHA-2 accelerator. Results indicate that the first symmetric HMAC authentication performs significantly faster than asymmetric verification in software, taking around $6\,\mathrm{ms}$. Signature verification of the proposed XMSS OTS is faster than ECDSA by a factor of 10, excluding the firmware hashing.

Bos et al. [104] utilize a secure boot mechanism that works similarly, verifying the PQ signature only once during bootstrapping and thereafter relying on a lighter reference proof. This speed-security tradeoff has to be evaluated on a case-by-case basis. Another option for tree height is the multitree approach. Kampanakis et al. [107] propose consolidating the keys involved in the UEFI secure boot use case into a single tree hierarchy. This approach can simplify key revocation and save resources. However, implementing it may be challenging due to the various entities involved in embedded platform development. Further research is needed to explore this approach and its potential use in update mechanisms.

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

Memory Constraints: As discussed, memory constraints can be prohibitive on embedded devices, especially on µCs. Marzougui et al. [103] chose the ARM Cortex-R5 as their embedded platform, which is more potent than the Cortex-M4 and is equipped with 512 kB RAM. Their software benchmark shows that XMSS with a tree height of 20, produces a smaller RAM footprint of less than 5 kB while qTesla uses around 50 kB. The Dilithium implementation by Bos et al. [104] uses 3 kB of the stack, which is, while being 10 times larger than Elliptic Curve Cryptography (ECC), small enough for the used S32G.

Using only the OTS also helps in reducing memory requirements. When comparing the OTS to regular XMSS, the authors observe a signature size reduction of 10 – 20 %, depending on the XMSS parameters [106]. The flash memory requirements are also lower than typical ECDSA implementations due to the small code size of the OTS validation, which offsets the larger signature size.

Hardware Support: As already touched on, in the algorithm class selection, general-purpose hash accelerators can significantly improve the performance of hash-based algorithms. Especially since SHA-2 hashes are widely used in embedded environments, these general-purpose accelerators are often found in embedded devices. Additionally, complete hardware FPGA implementations of XMSS and LMS are promising compared to those of classical algorithms. While dedicated FPGA hardware accelerators for a single signature algorithm are not common in IoT devices, they can be found in more specialized settings like industrial environments.

Wagner et al. [22], [108] explore hash-based PQ DSAs for embedded secure boot, focusing on SPHINCS+, LMS, and XMSS. They develop a hardware/software co-design using a newly designed hardware accelerator. Furthermore, they examine different parameter sets for stateful and stateless schemes using SHA-256. Most operations in hash-based algorithms occur during the hash chain calculation, while Merkle tree authentication accounts for less than 15 %. Thus, hardware acceleration for hash chain computations is crucial. However, using a standard SHA-256 hardware hash accelerator shifts the bottleneck to the communication with the accelerator due to the repeated hash calls. The authors address this by designing a new accelerator extending the OpenTitan by adding a chain register and digest feedback path to compute entire hash chains without requiring extra communication. Additionally, the authors benchmark different parameter configurations on the RISC-V Ibex processor. They test the parameters not only with their newly designed accelerator but also in software implementations and with a general-purpose SHA-256 core. General-purpose hash accelerators reduce computation time by around half for stateful algorithms and by a factor of five for SPHINCS+ compared to software implementations. The newly designed accelerator outperforms software and general-purpose SHA-256 core implementations by a significant margin, particularly in LMS, where it achieves nearly the same performance across all tested Winternitz parameters. Consequently, LMS with w = 256 allows for a fast verification while having the smallest signature when using the newly proposed accelerator.

Kampanakis et al. [107], [109] investigate SPHINCS+ and LMS for UEFI secure boot on high-end devices, as well as the performance of FPGA LMS implementations. Although their primary focus is not on embedded platforms, their research still provides relevant insights for constrained devices. They explore various parameter sets of SPHINCS+ and LMS. To increase the number of signatures per public key with LMS without significantly reducing signing performance, they opt for the HSS hypertree approach. This approach adds another Merkle tree on top of the first one, rather than increasing the tree's height. It enables combining the various keys required for secure boot into a single tree hierarchy and allows granular key revocation. The authors briefly consider using trees with more than two children per node but reject it since the memory and signature size benefits are negligible compared to the performance impact. Additionally, they implement LMS with a Winternitz parameter of eight on an FPGA and compare it to FPGA RSA implementations. The results show that LMS verification outperforms RSA-4096 while having a similar footprint on the FPGA. The authors suggest a tradeoff in FPGA LMS design between logic area and performance, and identify SHA-256 hash logic as the most significant optimization in terms of speed and size.

Kumar et al. [110] present another FPGA implementation targeting secure boot, focusing on XMSS. The target platform is a RISC-V-based Kintex7 FPGA, where the hardware XMSS implementation assists during the secure boot signature validation. As a parameter set, XMSS with SHA-256 and a tree height of ten is selected. The evaluation indicates that the XMSS implementation is comparable in size to ECDSA FPGA implementations, but verification is significantly slower, ranging from a factor of 10 to 30, depending on the level of parallelization. However, the verification process still takes less than two milliseconds in wall-clock time, which is acceptable for most embedded systems. The authors also compare their work to preliminary FPGA implementations of Dilithium, which show a smaller footprint and faster performance for XMSS.

3.2.3. Findings. In general, lattice-based algorithms offer better performance. In contrast, hash-based algorithms require less memory and offer a more robust cryptographic problem. This presents a tradeoff between RAM usage and verification speed. Size is the most constraining factor for PQC, as many µCs have only limited RAM and OTP memory is often not large enough for corresponding keys. Stateful algorithms are competitive with ECDSA in software, with LMS even outperforming it. Compared to stateful algorithms, SPHINCS+ performs worse and has a larger signature. Nevertheless, SPHINCS+ can be used as an alternative to LMS if secure state management cannot be guaranteed. Hardware acceleration has been shown to allow significant speed-ups for hash-based algorithms. However, in most cases, they do not seem strictly necessary to adopt PQ secure boot in embedded platforms.

Falcon might be interesting for further investigations into lattice-based algorithms in secure boot, due to its good performance and low RAM requirements for verification, paired with a reasonable signature size.

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

Table 7. PQ SECURE BOOT PAPERS, LISTING ALGORITHM, IMPLEMENTATION AND PLATFORM NEXT TO MAIN FINDINGS.

Reference	Approach	Algo.	Impl.	Platform	Main Findings
Marzougui et al. [103]	Identify requirements for embedded, PQ secure boot	XMSS, qTesla	Software	Cortex R5	- Main limitations are memory requirements and verification performance - Hash-based and lattice-based algorithms are promising
Wagner et al. [22]	Hardware/Software co-design for hash-based PQ secure boot	LMS, XMSS, SPHINCS+	Software, FPGA, ASIC	RISC-V Ibex, OpenTitan	- LMS outperforms XMSS and SPHINCS+ in hard- and software - LMS outperforms ECDSA in software - General-purpose SHA-2 accelerator boost verification
Kampanakis et al. [107]	SPHINCS+ and LMS for UEFI secure boot on different platforms	LMS, SPHINCS+	FPGA	Kintex UltraScale	- Hypertree LMS approach can combine different keys needed for UEFI secure boot - LMS faster verification than RSA in FPGA implementations
Kumar et al. [110]	XMSS FPGA hardware optimizations for secure boot	XMSS	FPGA	Kintex 7	- XMSS comparable to ECDSA in footprint size but slower verification on FPGA - XMSS smaller and faster than Dilithium in FPGAs
Román et al. [106]	Novel two-stage secure boot with PUF plus PQ DSAs	XMSS	Software	ESP32	- Initially verify complete XMSS signature on purchase; verify only OTS at boot for speed and size benefits - Significantly faster and smaller on storage compared to ECDSA
Bos et al. [104]	Dilithium based secure boot on an automotive S32G platform	Dilithium	Software	S32G	- Dilithium 2-10 times slower than RSA or ECDSA - But still feasible, especially if additional features like reference proof is used

3.3. Secure Software Updates

Secure updates are another crucial security aspect of embedded devices. Patches to the application software and firmware are vital to their security, given their long lifetimes. Securing these update processes closes the attack vector of infecting embedded platforms with inauthentic, malicious firmware. The IETF recognized this threat and founded the SUIT working group. The working group has published two RFCs [111], [112] to date, with multiple drafts currently in progress.

3.3.1. Secure Update Process. The SUIT workflow facilitates updates for IoT devices integrated into the secure boot RoT and allows crypto-agility with PQC. SUIT's primary security objective is authenticity, which is achieved through digests, signatures, and certificates. It protects against various attacks, such as malicious updates, replay attacks, and mismatch attacks. SUIT requires the device to be deployed with a trusted bootloader and a long-term public key. For new software updates, a digest is calculated, signed together with the corresponding metadata, and transferred via an update server to the applicable devices. On the device, the signature is verified, and the update is installed. Figure 4 visualizes the update flow. The security and performance requirements for the SUIT workflow are similar to those of the secure boot use case, with the main cryptographic operation being signature verification. However, instead of focusing on runtime performance, the SUIT workflow benefits most from small signatures and public keys, since the update duration is more influenced by network transfer costs than signature validation. Furthermore, the implementation size plays a significant role in the overall firmware size and, consequently, in updates that include the cryptographic library.

3.3.2. PQ Secure Updates Literature Analysis. Although secure updates and firmware signing are mentioned as one of the core applications of PQC [24], [26], there is surprisingly little research on this use case. Trusted firmware updates are mentioned in some of the secure boot papers [106], [107], but only from the perspective of integrating new key material.

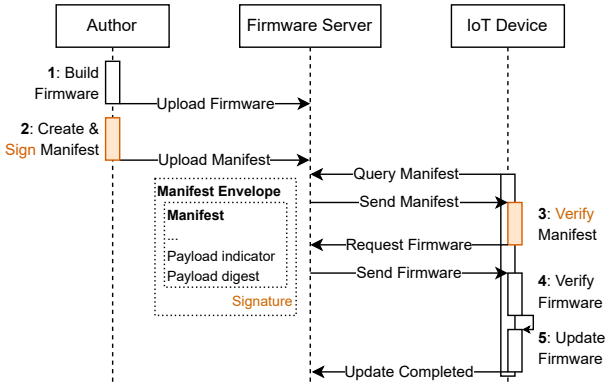


Figure 4. Exemplary SUIT workflow, the parts impacted by the PQC migration are highlighted in orange.

Benegas et al. [49] focus on PQC integration into SUIT for constrained embedded devices. Their target firmware is RIOT [113], an open-source IoT firmware with SUIT integration. The authors differentiate between updates of different sizes ranging from $5 - 250\,\mathrm{kB}$, including and excluding the PQ DSA library. They benchmark SUIT on a Cortex-M4, an ESP32, and a RISC-V development board. First, the authors investigate different hash functions for integrity and PQ signature checks. They identify no direct requirement for PQ SUIT but advise using SHA3-256 since it is used in most PQ DSAs, saving flash space and minimizing complexity. The choice of PQ DSA is more complicated and depends on multiple different factors. The authors compare ECDSA to LMS, Dilithium, and Falcon. They find that in small updates, the larger PQC signatures dominate, with Falcon performing best, due to its small signature and public key. LMS offers a good trade-off for larger updates containing the crypto library itself, as its implementation is smaller. They consider it to be the most secure option for a long-term deployment, such as the RoT. Table 8 lists a comparison between Falcon and LMS against ECDSA for a medium-sized update ($\approx 50\,\mathrm{kB}$). The authors also measure RAM requirements during verification and find that LMS is similar to ECDSA with Falcon being second. Their verification performance benchmarks show that PQ DSAs are

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

Table 8. RELATIVE COST INCREASE OF FALCON AND LMS FOR SUIT ON ARM CORTEX-M4 ADOPTED FROM BANEGAS ET AL. [49].

	Firmware on Flash	Stack Usage	Tranfer excl. Library	Transfer incl. Library
ECDSA	52.4 kB	16.3 kB	47 kB	53 kB
Falcon	+120 %	+18 %	+1.1 %	+120 %
LMS	+34 %	+1.2 %	+9 %	+43 %

faster than ECDSA on all embedded devices. However, verification typically has little impact on overall update performance, particularly with larger updates. For the largest update of 250 kB, the choice of PQ DSA has a negligible impact on overall performance, as network transfer is the bottleneck, and the choice of signature affects the total image size by only a few percent. Dilithium ranks second in both small updates, in terms of signature size overhead, and large updates, in terms of implementation size. However, it does require more stack memory than other PQC algorithms.

Bos et al. [114] analyze the applicability of PQC for automotive Over-the-Air (OTA) updates. For this, they apply Dilithium to the AUTOSAR update framework and evaluate the latency on a NXP S32G vehicle network processor, which consists of a Cortex-M7 and Cortex-A53 cores. Additionally, the board includes an HSE, which has been extended to include the capability to verify Dilithium signatures. They find in their benchmark that Dilithium does not introduce an overall significant latency to the update process, neither on the host processor nor on the HSE. While less flexible and still slower for now, they emphasize the importance of the HSE in protecting against side-channel and fault attacks. Bos et al. discuss the different signature sizes of PQ algorithms but do not evaluate their impact on the update process.

This coincides with the overall findings by La Manna et al. [115], who tested Dilithium, Falcon, ECDSA, and RSA for automotive updates. They evaluate each algorithm at the security levels of 128, 192, and 256 bit, except for Falcon, which does not provide a 192-bit level. As a testbed, they utilized a Cortex-A53 and ran three different sizes of typical software updates. Their first major finding is that, as update size increases, the PQ algorithms' verification time increases drastically, whereas the classical algorithm increases only slightly. Secondly, they found that the verification time of the PQ algorithms remains relatively constant across all security levels, whereas the time increases for classical algorithms. Both observations can be explained by the dominant influence of the different hash algorithms on the overall execution time. The difference is because classical algorithms use the hardware-accelerated SHA-256, while PQ uses SHAKE in software. Thus, the actual bottleneck is not the PQC itself, but rather the availability of hardware acceleration.

3.3.3. Findings. Secure firmware updates present a multidimensional challenge due to network transfer costs and the long-term key that forms the RoT. The size of the update during transfer can be affected not only by the PQ signature but also by the PQC implementation when the crypto library itself is included in the update. Banegas et al. [49] show a trade-off among the tested PQC algorithms. Falcon performs well in terms of signature size and verification speed, but has the largest implementation size. For larger updates that include the crypto libraries themselves, LMS seems to be the best option, since its larger signature has less impact on the required network bandwidth, while its implementation is relatively small. Additionally, LMS is less demanding on RAM-constrained devices, with moderate RAM usage similar to ECDSA. As with secure boot, the long lifetime of the key is a concern because the RoT cannot be easily updated. Since Falcon and Dilithium are both lattice-based algorithms with less cryptanalysis so far, early adopters may want to deploy hash-based algorithms first, given their conservative security assumptions. According to the secure boot analysis in Section 3.2.3 and the work by Campos et al. [102], LMS appears to perform better than XMSS on embedded devices and should be the first choice when employing stateful hash-based algorithms. Secure updates are another use case whose performance profits from hardware acceleration, even if verification time has less impact. And, just as with secure boot, it is crucial to have implementations protected against side-channel and fault attacks.

4. Conclusion

This survey provides the first overview of current research on PQC for embedded devices, to enable developers to implement quantum-resistant solutions. We observe that for most use cases, PQC is already competitive with classical cryptography, and implementation can be initiated within certain limits. However, most papers indicate that transitioning to PQC is not a straightforward drop-in replacement of algorithms and will come with a tradeoff between higher memory consumption and latency, especially with regards to PQ DSAs. Another point is the careful selection of the PQ DSA based on the lifetime of the public key. Lattice-based algorithms are currently perform well and are at the forefront of standardization. However, they are relatively new and may be affected by advances in cryptanalysis. On the other hand, hash-based algorithms are preferred for applications that rely on long-term public keys due to their well-understood security assumptions and high security levels. Hardware limitations can be a significant obstacle in embedded environments. Still, the analyzed research suggests that for devices similar to the ARM Cortex-M4 and better, software implementations are sufficient for all the investigated use cases. As the ARM Cortex-M4 is considered a mid-range product in the realm of embedded devices, it is unclear whether the presented results apply to devices with lower computational power and RAM constraints. Especially constraints in RAM or OTP memory may prove insurmountable for some algorithms.

Thus, future work should evaluate use cases on a broader, more realistic set of hardware. Especially, real-time critical systems, such as programmable logic controllers, need more study. The substantial performance impact of hardware accelerators warrants a deeper, systematic study of current approaches, strategies, and their impact on PQC use cases. Beyond TLS, secure boot, and updates, more use cases (e.g., VPN, remote access) need investigation in embedded PQC research. For these

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

cases, clearly defined benchmarks should be established to facilitate better comparison of results. Optimization strategies like KEMTLS or rapid XMSS should be pursued to enable further development. Furthermore, hybrid algorithms and their impact on embedded use cases should be explored as a potential migration strategy, particularly as they are promoted in some national strategies. Additionally, focus should be placed on the algorithms from the NIST Additional Digital Signature Schemes trial and the newly standardized HQC, as so far they have hardly been investigated for use cases in embedded systems. Implementations require continuous evaluation to prevent vulnerabilities, including side-channel or fault attacks.

With the current batch of standardized or to-be-standardized algorithms and NIST encouraging to begin the transition [7], the changeover to PQC is on the horizon. Current research supports this, showing that many use cases, albeit with some restrictions, are already feasible on embedded devices. Thus, developers should already begin developing their migration strategies to future-proof their products.

Acknowledgment

This manuscript was created with assistance by AI tools for editing, wording, and grammar enhancement. The project and research are supported by the Bavarian Ministry of Economic Affairs, Regional Development and Energy with funds from the Hightech Agenda Bayern.

References

[1] L. S. Vailshery, *Number of Internet of Things (IoT) connections worldwide from 2022 to 2023, with forecasts from 2024 to 2033*, Statista, 2024. [Online]. Available: https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/

[2] D. Papp, Z. Ma, and L. Buttyan, "Embedded systems security: Threats, vulnerabilities, and attack taxonomy", in *2015 13th Annual Conference on Privacy, Security and Trust (PST)*, 2015, pp. 145–152. DOI: 10.1109/PST.2015.7232966

[3] M. Antonakakis et al., "Understanding the mirai botnet", in *26th USENIX Security Symposium (USENIX Security 17)*, Vancouver, BC: USENIX Association, Aug. 2017, pp. 1093–1110, ISBN: 978-1-931971-40-9. [Online]. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/antonakakis

[4] D. Castelveccihi. "Ibm releases first-ever 1,000-qubit quantum chip", Accessed: Jan. 30, 2024. [Online]. Available: https://doi.org/10.1038/d41586-023-03854-1

[5] D. Castelveccihi. "Google's quantum computer hits key milestone by reducing errors", Accessed: Jan. 30, 2024. [Online]. Available: https://doi.org/10.1038/d41586-023-00536-w

[6] National Institute of Standards and Technology. "Information Technology Laboratory - Computer Security Resource Center: Post-Quantum Cryptography". [Online]. Available: https://www.nist.gov/pqcrypto

[7] National Institute of Standards and Technology. "Nist releases first 3 finalized post-quantum encryption standards". [Online]. Available: https://www.nist.gov/news-events/news/2024/08/nist-releases-first-3-finalized-post-quantum-encryption-standards

[8] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer", *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, Oct. 1997, ISSN: 1095-7111. DOI: 10.1137/s0097problem539795293172 [Online]. Available: http://dx.doi.org/10.1137/S0097539795293172

[9] National Institute of Standards and Technology, *Submission requirements and evaluation criteria for the post-quantum cryptography standardization process*, en, 2016. [Online]. Available: https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf

[10] National Institute of Standards and Technology. "Pqc standardization process: Announcing four candidates to be standardized, plus fourth round candidates". [Online]. Available: https://csrc.nist.gov/News/2022/pqc-candidates-to-be-standardized-and-round-4

[11] National Institute of Standards and Technology. "NIST Selects HQC as Fifth Algorithm for Post-Quantum Encryption". [Online]. Available: https://www.nist.gov/news-events/news/2025/03/nist-selects-hqc-fifth-algorithm-post-quantum-encryption

[12] National Institute of Standards and Technology. "Nist announces 14 candidates to advance to the second round of the additional digital signatures for the post-quantum cryptography standardization process", Accessed: Oct. 25, 2024. [Online]. Available: https://www.nist.gov/news-events/news/2024/10/nist-announces-14-candidates-advance-second-round-additional-digital

[13] E. Barker, L. Chen, and R. Davis, *Recommendation for key-derivation methods in key-establishment schemes*, en, 2020. DOI: https://doi.org/10.6028/NIST.SP.800-56Cr2

[14] F. D, "Terminology for Post-Quantum Traditional Hybrid Schemes", Internet Engineering Task Force, Internet-Draft draft-ietf-pquip-pqt-hybrid-terminology-02, Feb. 2024, Work in Progress, 16 pp. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-pquip-pqt-hybrid-terminology/02/

[15] D. McGrew, M. Curcio, and S. Fluhrer, *Leighton-Micali Hash-Based Signatures*, RFC 8554, Apr. 2019. DOI: 10.17487/RFC8554

[16] A. Huelsing, D. Butin, S.-L. Gazdag, J. Rijneveld, and A. Mohaisen, *XMSS: eXtended Merkle Signature Scheme*, RFC 8391, May 2018. DOI: 10.17487/RFC8391 [Online]. Available: https://www.rfc-editor.org/info/rfc8391

[17] National Institute of Standards and Technology, *Stateless hash-based digital signature standard*, 2024. DOI: https://doi.org/10.6028/NIST.FIPS.205

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

[18] National Institute of Standards and Technology, *Module-lattice-based key-encapsulation mechanism standard*, 2024. DOI: https://doi.org/10.6028/NIST.FIPS.203

[19] National Institute of Standards and Technology, *Module-lattice-based digital signature standard*, 2024. DOI: https://doi.org/10.6028/NIST.FIPS.204

[20] R. Perlner, *Fips 206 status update*, 2025. [Online]. Available: https://csrc.nist.gov/csrc/media/presentations/2025/fips-206-fn-dsa-(falcon)/images-media/fips_206-perlner_2.1.pdf

[21] G. Alagic et al., *Status report on the fourth round of the nist post-quantum cryptography standardization process*, Gaithersburg, MD, 2025.

[22] A. Wagner, F. Oberhansl, and M. Schink, "To be, or not to be stateful: Post-quantum secure boot using hash-based signatures", in *Proceedings of the 2022 Workshop on Attacks and Solutions in Hardware Security*, ser. ASHES'22, Los Angeles, CA, USA: Association for Computing Machinery, 2022, pp. 85–94, ISBN: 9781450398848. DOI: 10.1145/3560834.3563831 [Online]. Available: https://doi.org/10.1145/3560834.3563831

[23] P. Karl, J. Schupp, and G. Sigl, *The impact of hash primitives and communication overhead for hardware-accelerated sphincs+*, Cryptology ePrint Archive, Paper 2023/1767, https://eprint.iacr.org/2023/1767, 2023. [Online]. Available: https://eprint.iacr.org/2023/1767

[24] D. Cooper, D. Apon, Q. Dang, M. Davidson, M. Dworkin, and C. Miller, *Recommendation for stateful hash-based signature schemes*, en, 2020. DOI: 10.6028/NIST.SP.800-208

[25] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, *The sphincs+ signature framework*, Cryptology ePrint Archive, Paper 2019/1086, https://eprint.iacr.org/2019/1086, 2019. DOI: 10.1145/3319535.3363229 [Online]. Available: https://eprint.iacr.org/2019/1086

[26] G. Alagic et al., *Status report on the third round of the nist post-quantum cryptography standardization process*, en, Jul. 5, 2022. DOI: https://doi.org/10.6028/NIST.IR.8413 [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=934458

[27] M. Ajtai, "Generating hard instances of lattice problems (extended abstract)", in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, ser. STOC '96, Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, pp. 99–108, ISBN: 0897917855. DOI: 10.1145/237814.237838 [Online]. Available: https://doi.org/10.1145/237814.237838

[28] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography", in *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*, ser. STOC '05, Baltimore, MD, USA: Association for Computing Machinery, 2005, pp. 84–93, ISBN: 1581139608. DOI: 10.1145/1060590.1060603 [Online]. Available: https://doi.org/10.1145/1060590.1060603

[29] V. Lyubashevsky, "Fiat-shamir with aborts: Applications to lattice and factoring-based signatures", in *Advances in Cryptology – ASIACRYPT 2009*, M. Matsui, Ed., Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 598–616, ISBN: 978-3-642-10366-7.

[30] P.-A. Fouque et al., *Falcon: Fast-fourier lattice-based compact signatures over ntru*, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:231637439

[31] L. Ducas and T. Prest, *Fast fourier orthogonalization*, Cryptology ePrint Archive, Paper 2015/1014, https://eprint.iacr.org/2015/1014, 2015. [Online]. Available: https://eprint.iacr.org/2015/1014

[32] University of Waterloo and evolutionQ, Inc. (SIKE Team), *Sike foreword and postscript: Sike and sidh are insecure and should not be used*, 2022. [Online]. Available: https://csrc.nist.gov/csrc/media/Projects/post-quantum-cryptography/documents/round-4/history-pqc-round-4-updates.pdf

[33] C. Aguilar Melchor et al., *Hamming quasi-cyclic (hqc): Fourth round version*, 2024. [Online]. Available: https://pqc-hqc.org/doc/hqc-specification_2024-10-30.pdf

[34] Open Quantum Safe. "Algorithms", Accessed: Jan. 14, 2025. [Online]. Available: https://openquantumsafe.org/liboqs/algorithms/

[35] S. Turner, D. Brown, K. Yiu, R. Housley, and T. Polk, *Rfc 5480: Elliptic curve cryptography subject public key information*, 2009. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5480.html

[36] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rush, *Rfc 8017: Pkcs #1: Rsa cryptography specifications version 2.2*, 2016. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8017

[37] GitHub. "Pqclean/pqclean: Clean, portable, tested implementations of post-quantum cryptography", Accessed: Jan. 14, 2025. [Online]. Available: https://github.com/PQClean/PQClean?tab=readme-ov-file

[38] GitHub. "Pq-code-package: Pq code package: Building high-assurance implementations of post-quantum cryptography algorithms", Accessed: Mar. 25, 2026. [Online]. Available: https://github.com/pq-code-package

[39] GitHub. "Mupq/pqm4: Post-quantum crypto library for the arm cortex-m4", Accessed: Jan. 14, 2025. [Online]. Available: https://github.com/mupq/pqm4

[40] GitHub. "Open-quantum-safe/liboqs: C library for prototyping and experimenting with quantum-resistant cryptography", Accessed: Jan. 14, 2025. [Online]. Available: https://github.com/open-quantum-safe/liboqs

[41] GitHub. "Open-quantum-safe/oqs-provider: Openssl 3 provider containing post-quantum algorithms", Accessed: Jan. 14, 2025. [Online]. Available: https://github.com/open-quantum-safe/oqs-provider

[42] Open Quantum Safe. "Tls", Accessed: Jan. 14, 2025. [Online]. Available: https://openquantumsafe.org/applications/tls.html

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

[43] WolfSSL. "G. experimenting with post-quantum cryptography - wolfssl manual", Accessed: Jan. 14, 2025. [Online]. Available: https://www.wolfssl.com/documentation/manuals/wolfssl/appendix07.html

[44] G. Tasopoulos, C. Dimopoulos, A. P. Fournaris, R. K. Zhao, A. Sakzad, and R. Steinfeld, "Energy consumption evaluation of post-quantum tls 1.3 for resource-constrained embedded devices", *International Conference on Computing Frontiers (CF)*, vol. 20th, 2023. DOI: 10.1145/3587135.3592821

[45] G. Tasopoulos, J. Li, A. P. Fournaris, R. K. Zhao, A. Sakzad, and R. Steinfeld, "Performance evaluation of post-quantum tls 1.3 on resource-constrained embedded systems", in *Information Security Practice and Experience*, C. Su, D. Gritzalis, and V. Piuri, Eds., Cham: Springer International Publishing, 2022, pp. 432–451, ISBN: 978-3-031-21280-2. DOI: 10.1007/978-3-031-21280-2_24

[46] S. Paul, "On the transition to post-quantum cryptography in the industrial internet of things", en, Ph.D. dissertation, Technische Universität Darmstadt, Darmstadt, 2022, xxix, 188 Seiten. DOI: https://doi.org/10.26083/tuprints-00021368 [Online]. Available: http://tuprints.ulb.tu-darmstadt.de/21368/

[47] K. Bürstinghaus-Steinbach, C. Krauß, R. Niederhagen, and M. Schneider, *Post-quantum tls on embedded systems*, Cryptology ePrint Archive, Paper 2020/308, 2020. DOI: 10.1145/3320269.3384725 [Online]. Available: https://eprint.iacr.org/2020/308

[48] L. Reichert, N. Coppik, and S. Finster, "Performance evaluation of quantum-resistant algorithms on industrial embedded systems", *Availability, Reliability and Security (ARES)*, pp. 127–148, 2025. DOI: 10.1007/978-3-032-00630-1_8

[49] G. Banegas, K. Zandberg, E. Baccelli, A. Herrmann, and B. Smith, "Quantum-resistant software update security on low-power networked embedded devices", in *Applied Cryptography and Network Security*, G. Ateniese and D. Venturi, Eds., Cham: Springer International Publishing, 2022, pp. 872–891, ISBN: 978-3-031-09234-3. DOI: 10.1007/978-3-031-09234-3_43

[50] B. Halak, T. Gibson, M. Henley, C.-B. Botea, B. Heath, and S. Khan, "Evaluation of performance, energy, and computation costs of quantum-attack resilient encryption algorithms for embedded devices", *IEEE Access*, vol. 12, pp. 8791–8805, 2024. DOI: 10.1109/ACCESS.2024.3350775

[51] D. Marchsreiter and J. Sepúlveda, "Hybrid post-quantum enhanced tls 1.3 on embedded devices", in *2022 25th Euromicro Conference on Digital System Design (DSD)*, 2022, pp. 905–912. DOI: 10.1109/DSD57027.2022.00127

[52] D.-T. Dam, T.-H. Tran, V.-P. Hoang, C.-K. Pham, and T.-T. Hoang, "A survey of post-quantum cryptography: Start of a new race", *Cryptography*, vol. 7, no. 3, 2023, ISSN: 2410-387X. DOI: 10.3390/cryptography7030040 [Online]. Available: https://www.mdpi.com/2410-387X/7/3/40

[53] B. Koziel, M. M. Kermani, and R. Azarderakhsh, "Post-quantum cryptographic hardware and embedded systems", in *Emerging Topics in Hardware Security*, M. Tehranipoor, Ed. Cham: Springer International Publishing, 2021, pp. 229–255, ISBN: 978-3-030-64448-2. DOI: 10.1007/978-3-030-64448-2_9 [Online]. Available: https://doi.org/10.1007/978-3-030-64448-2_9

[54] A. Mosally, I. Kabir, and M. Hammoudeh, "Quantum computing: A journey into the next frontier of information and communication security", in CRC Press, 2024, ch. Post-Quantum Cryptography in Embedded Systems, Future Trends, and Research Directions, ISBN: 9781003475286. DOI: 10.1201/9781003475286

[55] K.-A. Shim, "On the suitability of post-quantum signature schemes for internet of things", *IEEE Internet of Things Journal*, vol. 11, no. 6, pp. 10 648–10 665, 2024. DOI: 10.1109/JIOT.2023.3327400

[56] N. Alnahawi et al., *On the state of post-quantum cryptography migration*, INFORMATIK 2021, 2021. DOI: 10.18420/informatik2021-078

[57] A. Banerjee, T. Reddy.K, D. Schoinianakis, T. Hollebeek, and M. Ounsworth, "Post-Quantum Cryptography for Engineers", Internet Engineering Task Force, Internet-Draft draft-ietf-pquip-pqc-engineers-14, Aug. 2025, Work in Progress, 50 pp. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-pquip-pqc-engineers/14/

[58] T. Reddy.K, D. Wing, B. S, and K. Kwiatkowski, "Adapting Constrained Devices for Post-Quantum Cryptography", Internet Engineering Task Force, Internet-Draft draft-ietf-pquip-pqc-hsm-constrained-05, Apr. 2026, Work in Progress, 27 pp. [Online]. Available: https://datatracker.ietf.org/doc/draft-ietf-pquip-pqc-hsm-constrained/05/

[59] L. Malina, L. Popelova, P. Dzurenda, J. Hajny, and Z. Martinasek, "On feasibility of post-quantum cryptography on small devices", *IFAC-PapersOnLine*, vol. 51, no. 6, pp. 462–467, 2018. DOI: 10.1016/j.ifacol.2018.07.104

[60] A. Khalid, S. McCarthy, M. O'Neill, and W. Liu, "Lattice-based cryptography for iot in a quantum world: Are we ready?", *International Workshop on Advances in Sensors and Interfaces (IWASI)*, vol. 8th, pp. 194–199, 2019. DOI: 10.1109/IWASI.2019.8791343

[61] T. M. Fernández-Caramés, "From pre-quantum to post-quantum iot security: A survey on quantum-resistant cryptosystems for the internet of things", *IEEE Internet of Things Journal*, vol. 7, no. 7, pp. 6457–6480, 2020. DOI: 10.1109/JIOT.2019.2958788

[62] B. Hession and J.-P. Kaps, "Feasibility and performance of pqc algorithms on microcontrollers", in *Second PQC Standardization Conference*, 2019. [Online]. Available: https://csrc.nist.gov/CSRC/media/Events/Second-PQC-Standardization-

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

Conference / documents / accepted - papers / kaps - feasibility-performance-pqc.pdf

[63] L. Palmer and Y. Fazea, "Lattice-based cryptography for internet-of-things in post-quantum computing", *Advances in Intelligent Computing Techniques and Applications (IRICT)*, pp. 233–246, 2024. DOI: 10.1007/978-3-031-59711-4{\textunderscore}21

[64] J. Howe and B. Westerbaan, *Benchmarking and analysing the nist pqc lattice-based signature schemes standards on the arm cortex m7*, Cryptology ePrint Archive, Paper 2022/405, https://eprint.iacr.org/2022/405, 2022. DOI: 10.1007/978-3-031-37679-5_19 [Online]. Available: https://eprint.iacr.org/2022/405

[65] M. EI-Hadedy, P. V. Ankunda, J. Ung, and W.-M. Hwu, "Securing the internet of medical things (iomt) with k3s and hybrid cryptography: Integrating post-quantum approaches for enhanced embedded system security", *Dallas Circuits and Systems Conference (DCAS)*, vol. 17th, pp. 1–6, 2024. DOI: 10.1109/DCAS61159.2024.10539887

[66] G. B. Satrya, Y. M. Agus, and A. B. Mnaouer, "A comparative study of post-quantum cryptographic algorithm implementations for secure and efficient energy systems monitoring", *Electronics*, vol. 12, no. 18, p. 3824, 2023. DOI: 10.3390/electronics12183824

[67] L. Malina, P. Dobias, P. Dzurenda, and G. Srivastava, "Quantum-resistant and secure mqtt communication", in *Proceedings of the 19th International Conference on Availability, Reliability and Security*, ser. ARES '24, Vienna, Austria: Association for Computing Machinery, 2024, ISBN: 9798400717185. DOI: 10.1145/3664476.3670463 [Online]. Available: https://doi.org/10.1145/3664476.3670463

[68] S. Paul, P. Scheible, and F. Wiemer, "Towards post-quantum security for cyber-physical systems: Integrating pqc into industrial m2m communication1", *Journal of Computer Security*, vol. 30, no. 4, pp. 623–653, 2022. DOI: 10.3233/JCS-210037 eprint: https://doi.org/10.3233/JCS-210037. [Online]. Available: https://doi.org/10.3233/JCS-210037

[69] R. Aissaoui, J.-C. Deneuville, C. Guerber, and A. Pirovano, "Authenticating civil uav communications with post-quantum digital signatures", *Digital Avionics Systems Conference (DASC)*, vol. 42nd, pp. 1–9, 2023. DOI: 10.1109/DASC58513.2023.10311111

[70] J. Choi, S. Yoon, and S. C. Seo, "Optimized falcon verify on cortex-m4 for post-quantum secure uav communications", *ICT Express*, 2024. DOI: 10.1016/j.icte.2024.11.002

[71] N. Alnahawi, N. Schmitt, A. Wiesmaier, and C.-M. Zok, "Towards next generation quantum-safe eids and emrtds – a survey", *ACM Trans. Embed. Comput. Syst.*, 2023, Just Accepted, ISSN: 1539-9087. DOI: 10.1145/3585517 [Online]. Available: https://doi.org/10.1145/3585517

[72] T. Fritzmann, J. Vith, D. Flórez, and J. Sepúlveda, "Post-quantum cryptography for automotive systems", *Microprocessors and Microsystems*, vol. 87, p. 104379, 2021. DOI: 10.1016/j.micpro.2021.104379

[73] K.-A. Shim, "A survey on post-quantum public-key signature schemes for secure vehicular communications", *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 9, pp. 14025–14042, 2022. DOI: 10.1109/TITS.2021.3131668

[74] P. Jungklass et al., "Crypto-agility in automotive real-time systems in context of post-quantum-cryptography", *escar Europe*, vol. 23rd, 2025.

[75] A. Rathweg, A. Osman, A. Fleig, E. Katsigianni, and A. Tschache, "Impacts of post-quantum cryptography on automotive security", *escar Europe*, vol. 21th, 2024.

[76] J. Hermelink, T. Pöppelmann, M. Stöttinger, Y. Wang, and Y. Wan, "Quantum safe authenticated key exchange protocol for automotive application", *escar Europe*, vol. 18th, 2020. DOI: 10.13154/294-7549

[77] Google. "Https encryption on the web – google transparency report", Accessed: Jan. 15, 2025. [Online]. Available: https://transparencyreport.google.com/https/overview

[78] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Assessing the overhead of post-quantum cryptography in tls 1.3 and ssh", *International Conference on emerging Networking Experiments and Technologies (CoNEXT)*, vol. 16th, 2020. DOI: 10.1145/3386367.3431305

[79] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-quantum authentication in tls 1.3: A performance study", *Network and Distributed System Security Symposium*, 2020. DOI: 10.14722/ndss.2020.24203

[80] M. Sosnowski et al., "The performance of post-quantum tls 1.3", *International Conference on emerging Networking Experiments and Technologies (CoNEXT)*, vol. 19th, pp. 19–27, 2023. DOI: 10.1145/3624354.3630585

[81] M. Raavi, P. Chandramouli, S. Wuthier, X. Zhou, and S.-Y. Chang, "Performance characterization of post-quantum digital certificates", in *2021 International Conference on Computer Communications and Networks (ICCCN)*, 2021, pp. 1–9. DOI: 10.1109/ICCCN52240.2021.9522179

[82] P. Kampanakis and W. Childs-Klein, "The impact of data-heavy, post-quantum tls 1.3 on the time-to-last-byte of real-world connections", *Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb)*, 2024. DOI: 10.14722/madweb.2024.23010

[83] N. Alnahawi, J. Müller, J. Oupický, and A. Wiesmaier, "A comprehensive survey on post-quantum tls", *IACR Communications in Cryptology (IACR CiC)*, vol. 1, 2024. DOI: 10.62056/ahee0iuc

[84] D. Mankowski, T. Wiggers, and V. Moonsamy, "Tls → post-quantum tls: Inspecting the tls landscape for pqc adoption on android", *European Symposium on Security and Privacy Workshops (EuroS&PW)*, pp. 526–538, 2023. DOI: 10.1109/EuroSPW59978.2023.00065

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

[85] Arduino, *Arduino nano*, 2025. [Online]. Available: https://docs.arduino.cc/resources/datasheets/A000005-datasheet.pdf

[86] STMicroelectronics, *Datasheet - stm32l4s5xx stm32l4s7xx stm32l4s9xx - ultra-low-power arm® cortex®-m4 32-bit mcu+fpu, 150dmips, up to 2mb flash, 640kb sram, lcd-tft & mipi dsi, aes+hash*, 2020. [Online]. Available: https://www.st.com/resource/en/datasheet/stm32l4r5zi.pdf

[87] Espressif, *Esp32-wroom-32e esp32-wroom-32ue datasheet: Version 1.7*, 2024. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32e_esp32-wroom-32ue_datasheet_en.pdf

[88] Raspberry Pi Ltd, *Raspberry pi 3 model b+*, 2023. [Online]. Available: https://datasheets.raspberrypi.com/rpi3/raspberry-pi-3-b-plus-product-brief.pdf

[89] Raspberry Pi Ltd, *Raspberry pi 4 model b*, 2024. [Online]. Available: https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-product-brief.pdf

[90] D. Moody. "Nist focus on the cortex m4, slide 22", Accessed: Jan. 30, 2024. [Online]. Available: https://csrc.nist.gov/CSRC/media/Presentations/the-2nd-round-of-the-nist-pqc-standardization-proc/images-media/moody-opening-remarks.pdf

[91] STMicroelectronics. "Arm® cortex®-m4 in a nutshell". [Online]. Available: https://www.st.com/content/st_com/en/arm-32-bit-microcontrollers/arm-cortex-m4.html

[92] SSLInsights. "11+ latest ssl certificates statistics january 2025: Tls stats", Accessed: Jan. 18, 2025. [Online]. Available: https://sslinsights.com/ssl-certificates-statistics/

[93] M. Schöffel, F. Lauer, C. C. Rheinländer, and N. Wehn, "Secure iot in the era of quantum computers-where are the bottlenecks?", *Sensors*, vol. 22, no. 7, 2022, ISSN: 1424-8220. DOI: 10.3390/s22072484 [Online]. Available: https://www.mdpi.com/1424-8220/22/7/2484

[94] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, Aug. 2018. DOI: 10.17487/RFC8446 [Online]. Available: https://www.rfc-editor.org/info/rfc8446

[95] S. Paul, Y. Kuzovkova, N. Lahr, and R. Niederhagen, "Mixed certificate chains for the transition to post-quantum authentication in tls 1.3", in *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, ser. ASIA CCS '22, Nagasaki, Japan: Association for Computing Machinery, 2022, pp. 727–740, ISBN: 9781450391405. DOI: 10.1145/3488932.3497755 [Online]. Available: https://doi.org/10.1145/3488932.3497755

[96] Y. Hanna, J. Bozhko, S. Tonyali, R. Harrilal-Parchment, M. Cebe, and K. Akkaya, "A comprehensive and realistic performance evaluation of post-quantum security for consumer iot devices", *Internet of Things*, vol. 33, p. 101 650, 2025.

[97] R. Gonzalez and T. Wiggers, "Kemtls vs. post-quantum tls: Performance on embedded systems", in *Security, Privacy, and Applied Cryptography Engineering*, L. Batina, S. Picek, and M. Mondal, Eds., Cham: Springer Nature Switzerland, 2022, pp. 99–117, ISBN: 978-3-031-22829-2.

[98] M. Schöffel, F. Lauer, C. C. Rheinländer, and N. Wehn, "On the energy costs of post-quantum kems in tls-based low-power secure iot", *International Conference on Internet-of-Things Design and Implementation (IoTDI '21)*, pp. 158–168, 2021. DOI: 10.1145/3450268.3453528

[99] D. Kern, C. Krauß, T. Lauser, N. Alnahawi, A. Wiesmaier, and R. Niederhagen, "Quantumcharge: Post-quantum cryptography for electric vehicle charging", *Applied Cryptography and Network Security (ACNS)*, vol. 13906, pp. 85–111, 2023. DOI: 10.1007/978-3-031-33491-7{\textunderscore}4

[100] P. Schwabe, D. Stebila, and T. Wiggers, "Post-quantum TLS without handshake signatures", in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '20, Virtual Event, USA: Association for Computing Machinery, 2020, pp. 1461–1480, ISBN: 9781450370899. DOI: 10.1145/3372297.3423350 [Online]. Available: https://kemtls.org/publication/kemtls/

[101] D. McGrew, P. Kampanakis, S. Fluhrer, S.-L. Gazdag, D. Butin, and J. Buchmann, "State management for hash-based signatures", in *Security Standardisation Research*, L. Chen, D. McGrew, and C. Mitchell, Eds., Cham: Springer International Publishing, 2016, pp. 244–260, ISBN: 978-3-319-49100-4.

[102] F. Campos, T. Kohlstadt, S. Reith, and M. Stöttinger, "Lms vs xmss: Comparison of stateful hash-based signature schemes on arm cortex-m4", in *Progress in Cryptology - AFRICACRYPT 2020*, A. Nitaj and A. Youssef, Eds., Cham: Springer International Publishing, 2020, pp. 258–277, ISBN: 978-3-030-51938-4.

[103] S. Marzougui and J. Krämer, "Post-quantum cryptography in embedded systems", in *Proceedings of the 14th International Conference on Availability, Reliability and Security*, ser. ARES '19, Canterbury, CA, United Kingdom: Association for Computing Machinery, 2019, ISBN: 9781450371643. DOI: 10.1145/3339252.3341475 [Online]. Available: https://doi.org/10.1145/3339252.3341475

[104] J. W. Bos, B. Carlson, J. Renes, M. Rotaru, A. Sprenkels, and G. P. Waters, "Post-quantum secure boot on vehicle network processors", *escar Europe*, vol. 20th, 2022. DOI: 10.13154/294-9372

[105] J. W. Bos, A. Hülsing, J. Renes, and C. van Vredendaal, *Rapidly verifiable xmss signatures*, Cryptology ePrint Archive, Paper 2020/898, https://eprint.iacr.org/2020/898, 2020. [Online]. Available: https://eprint.iacr.org/2020/898

[106] R. Román and I. Baturone, "A quantum-resistant and fast secure boot for iot devices using hash-based signatures and sram pufs", in *The Fifth International Conference on Safety and Security with IoT*, A. Nayyar, A. Paul, and S. Tanwar, Eds., Cham: Springer International Publishing, 2023, pp. 121–136, ISBN: 978-3-030-94285-4.

[107] P. Kampanakis, P. Panburana, M. Curcio, C. Shroff, and M. M. Alam, *Post-quantum lms and*

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

sphincs+ hash-based signatures for uefi secure boot, Cryptology ePrint Archive, Paper 2021/041, https://eprint.iacr.org/2021/041, 2021. [Online]. Available: https://eprint.iacr.org/2021/041

[108] A. Wagner, F. Oberhansl, and M. Schink, "Extended version—to be, or not to be stateful: Post-quantum secure boot using hash-based signatures", *Journal of Cryptographic Engineering*, vol. 14, no. 4, pp. 631–648, 2024. DOI: 10.1007/s13389-024-00362-4

[109] P. Kampanakis, P. Panburana, M. Curcio, and C. Shroff, "Post-quantum hash-based signatures for secure boot", *Silicon Valley Cybersecurity Conference (SVCC)*, vol. 1383, pp. 71–86, 2021. DOI: 10.1007/978-3-030-72725-3{\textunderscore}5

[110] V. B. Y. Kumar, N. Gupta, A. Chattopadhyay, M. Kasper, C. Krauß, and R. Niederhagen, "Post-quantum secure boot", in *2020 Design, Automation & Test in Europe Conference & Exhibition*, 2020, pp. 1582–1585. DOI: 10 . 23919 / DATE48585.2020.9116252

[111] B. Moran, H. Tschofenig, D. Brown, and M. Meriac, *A Firmware Update Architecture for Internet of Things*, RFC 9019, Apr. 2021. DOI: 10.17487/RFC9019 [Online]. Available: https://www.rfc-editor.org/info/rfc9019

[112] B. Moran, H. Tschofenig, and H. Birkholz, *A Manifest Information Model for Firmware Updates in Internet of Things (IoT) Devices*, RFC 9124, Jan. 2022. DOI: 10.17487/RFC9124 [Online]. Available: https://www.rfc-editor.org/info/rfc9124

[113] GitHub. "Riot-os/riot: Riot - the friendly os for iot", Accessed: Jan. 19, 2025. [Online]. Available: https://github.com/RIOT-OS/RIOT

[114] J. W. Bos, A. Dima, A. Kiening, and J. Renes, "Post-quantum secure over-the-air update of automotive systems", *escar Europe*, vol. 21st, 2023. DOI: 10.13154/294-10380

[115] M. La Manna, P. Perazzo, L. Treccozzi, and G. Dini, "Assessing the cost of quantum security for automotive over -the-air updates", *IEEE Transactions on Intelligent Transportation Systems*, pp. 1–6, 2021. DOI: 10.1109/ISCC53001.2021.9631426

[116] National Institute of Standards and Technology. "Post-quantum cryptography: Security (evaluation criteria)", Accessed: Nov. 16, 2025. [Online]. Available: https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/evaluation-criteria/security-(evaluation-criteria)

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.

Appendix A.

A.1. Performance Dataset

The performance dataset used to generate Figure 1 and Figure 5 can be found at https://www.doi.org/10.5281/zenodo.19896178.

A.2. NIST Algorithms Security Level

Table 9. NIST SECURITY LEVEL REQUIREMENTS, THEIR SECURITY ANALOGOUS AND CORRESPONDING ALGORITHM VERSIONS [26], [116].

SL	Security Analogous	Kyber	HQC	Dilithium	Falcon	SPHINCS+
1	AES-128, key search	Kyber512	HQC-128	-	Falcon-512	SPHINCS+-128s/f
2	SHA-256, collision search	-	-	Dilithium2	-	-
3	AES-192, key search	Kyber668	HQC-192	Dilithium3	-	SPHINCS+-192s/f
4	SHA-384, collision search	-	-	-	-	-
5	AES-256, key search	Kyber1024	HQC-256	Dilithium5	Falcon-1024	SPHINCS+-256s/f

A.3. PQ TLS Comparison

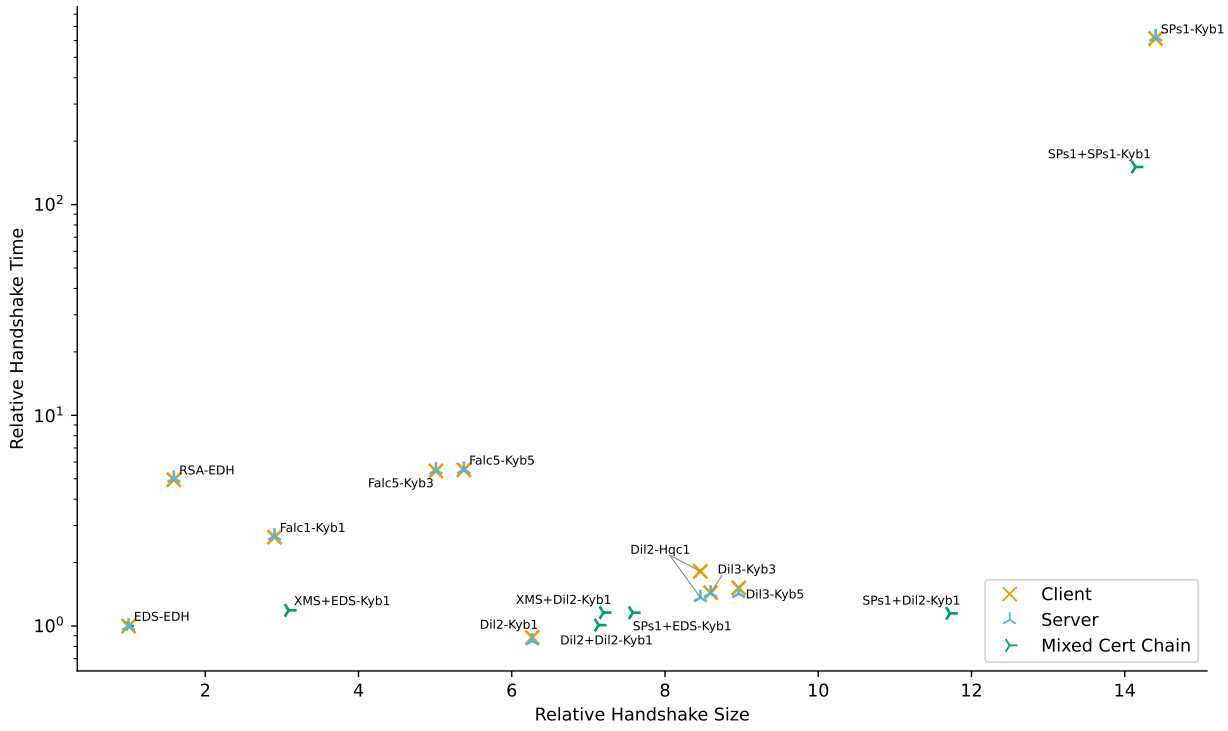


Figure 5. Comparison of different PQC algorithms for TLS. While the time to complete the handshake is comparable to classic algorithms, the relative communication size increases due to the larger signatures. Results are normalized to ECDSA and based on data from [45] and [95] for mixed certificate chains.

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 15:53:17 UTC from IEEE Xplore. Restrictions apply.